CODE4LIFE TECHNOLOGY

LƯU LẠI ĐỂ ÔN THI

CẨM NANG MẠNG

MÁY TÍNH CHO SINH VIÊN VIỆT NAM • 107 TRANG

```python
import ipaddress as ip

net = ip.ip_network("192.168.1.64/26")
print(net.netmask, net.num_addresses)
print(ip.ip_address("192.168.1.130") in net)
```

Chung dải 192.168.1 mà khác mạng. Vì sao?

OSI · Subnet · Định tuyến · TCP · Ôn thi

Bản đồ cẩm nang

Đi từ tổng quan, qua từng tầng, tới thực hành và ôn thi.

1 TỔNG QUAN Trang 3

2 TẦNG VẬT LÝ Trang 17

3 TẦNG LIÊN KẾT DỮ LIỆU Trang 23

4 TẦNG MẠNG VÀ ĐỊA CHỈ IP Trang 35

5 ĐỊNH TUYẾN Trang 50

6 TẦNG GIAO VẬN Trang 58

7 TẦNG ỨNG DỤNG Trang 71

8 AN TOÀN MẠNG Trang 78

9 THỰC HÀNH Trang 84

10 ÔN THI Trang 92

Dùng cẩm nang này thế nào?

Mỗi trường có đề cương riêng, nhưng phần lõi của môn Mạng máy tính khá giống nhau. Cẩm nang bám theo phần lõi đó.

Mục tiêu của bạn	Nên đọc
Học môn Mạng máy tính	**Phần 1 đến 7**
Thi subnet, định tuyến	**Phần 4, 5, 10**
Thực hành Packet Tracer	**Phần 9**
An toàn mạng	**Phần 8**
Chuẩn bị CCNA	**Phần 3, 4, 5, 8, 9**

Luôn đối chiếu với đề cương của trường bạn. Thầy cô có thể đổi tên chương, trình tự, hoặc dùng cách học từ trên xuống.

Nội dung được đối chiếu với đề cương IT3080 của ĐH Bách khoa Hà Nội, các tài liệu môn học của ĐH Bách khoa TP.HCM, UIT, PTIT, và khung kiến thức CCNA 200-301.

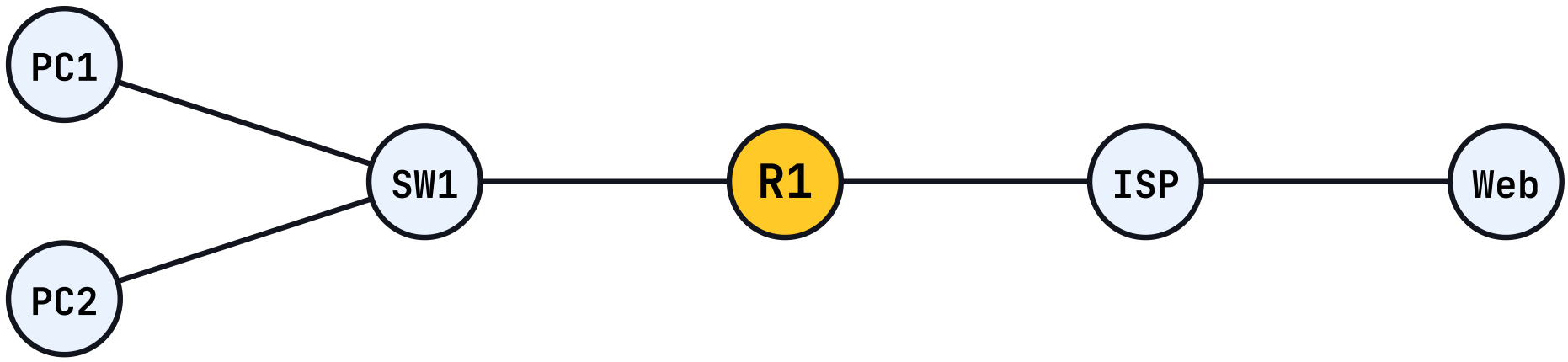
Một khái niệm, nhiều tên gọi

Giáo trình mỗi nơi dịch một kiểu. Đề thi đổi tên thì đừng hoảng.

Tiếng Anh	Các tên hay gặp
Transport layer	Tầng giao vận, tầng vận chuyển
Data link layer	Tầng liên kết dữ liệu
Presentation layer	Tầng trình diễn, tầng trình bày
Routing	Định tuyến, chọn đường
Forwarding	Chuyển tiếp
Switching	Chuyển mạch
Frame / Packet / Segment	Khung / Gói tin / Đoạn
Flow control	Kiểm soát luồng, điều khiển luồng
Congestion control	Kiểm soát tắc nghẽn
Multiplexing	Dồn kênh, ghép kênh
Encapsulation	Đóng gói
Subnet mask	Mặt nạ mạng con
Default gateway	Cổng mặc định
Bandwidth	Băng thông

1 Mạng máy tính là gì?

Là tập hợp các thiết bị nối với nhau để trao đổi dữ liệu theo những quy tắc chung gọi là giao thức.



Thành phần	Vai trò	Ví dụ
Thiết bị đầu cuối	Tạo và nhận dữ liệu	Laptop, điện thoại, server
Thiết bị trung gian	Chuyển dữ liệu đi tiếp	Switch, router, access point
Đường truyền	Mang tín hiệu	Cáp đồng, cáp quang, sóng vô tuyến
Giao thức	Quy định định dạng, thứ tự, cách xử lý	IP, TCP, HTTP

Giao thức giống ngôn ngữ chung: hai bên phải hiểu cùng một quy tắc mới nói chuyện được.

2 Phân loại mạng

Loại	Phạm vi	Ví dụ
PAN	Quanh một người	Tai nghe Bluetooth nối điện thoại
LAN	Một tòa nhà, khuôn viên	Mạng phòng máy, ký túc xá
MAN	Một thành phố	Mạng nối các cơ sở của trường
WAN	Quốc gia, toàn cầu	Mạng của nhà mạng, Internet

CLIENT-SERVER
Máy khách gửi yêu cầu, máy chủ phục vụ. Dễ quản lý, nhưng máy chủ có thể quá tải.

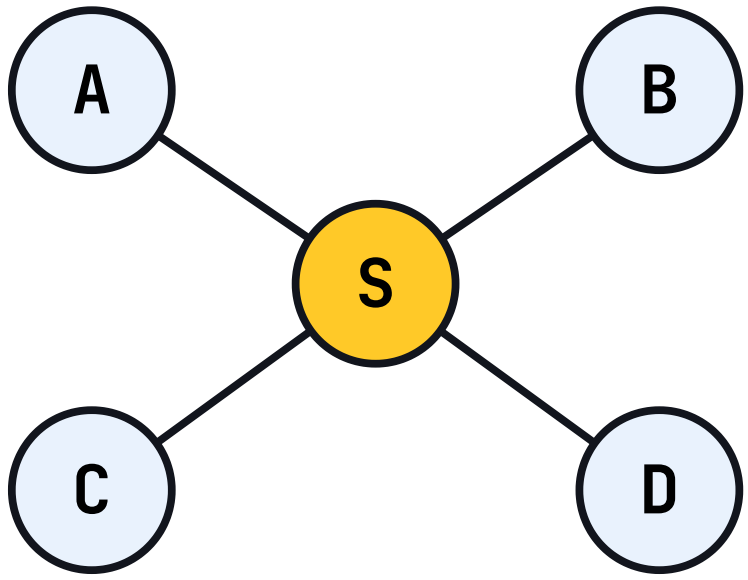
NGANG HÀNG (P2P)
Mỗi máy vừa nhận vừa cho. Dễ mở rộng, nhưng khó quản lý và bảo mật.

CHUYỂN MẠCH KÊNH
Giữ riêng một đường suốt cuộc gọi. Chất lượng ổn định, nhưng lãng phí khi im lặng.

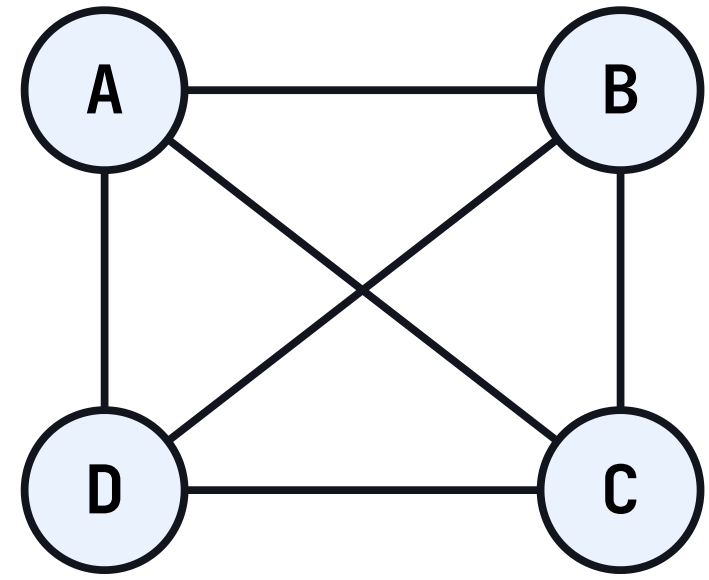
CHUYỂN MẠCH GÓI
Chia dữ liệu thành gói, dùng chung đường truyền. Hiệu quả, nhưng có thể trễ và mất gói.

Internet dùng chuyển mạch gói.

3 Topology: cách nối mạng



Hình sao (star)



Lưới đầy đủ (full mesh)

Kiểu	Ưu điểm	Nhược điểm
Bus	Ít cáp	Đứt trục chính là hỏng cả mạng
Star	Dễ thêm máy, dễ tìm lỗi	Hỏng thiết bị trung tâm là mất mạng
Ring	Dữ liệu đi theo vòng có trật tự	Một điểm hỏng có thể ảnh hưởng cả vòng
Mesh	Nhiều đường dự phòng	Tốn cáp: n máy cần n(n – 1)/2 đường
Tree	Dễ phân cấp, mở rộng	Phụ thuộc các nút phía trên

LAN ngày nay chủ yếu là hình sao mở rộng: máy nối vào switch, các switch nối lên switch trung tâm.

4 Internet và Việt Nam

Internet là mạng của các mạng. Các nhà mạng nối với nhau bằng hợp đồng trung chuyển hoặc tại các điểm trao đổi.

Thành phần	Ý nghĩa
ISP	Nhà cung cấp dịch vụ Internet
IXP	Điểm các mạng gặp nhau để trao đổi lưu lượng
AS	Một mạng có chính sách định tuyến riêng, có số AS riêng
Cáp quang biển	Đường trục nối các quốc gia
CDN	Đặt nội dung gần người dùng để tải nhanh hơn

Ở VIỆT NAM
VNNIC quản lý tên miền .vn, tài nguyên địa chỉ IP, số hiệu mạng và vận hành trạm trung chuyển Internet quốc gia VNIX.

Số liệu công bố ngày 19/6/2026: tỷ lệ dùng IPv6 của Việt Nam đạt 67,68%, đứng thứ 7 thế giới và thứ 2 Đông Nam Á.

Nguồn: Cổng Thông tin điện tử Chính phủ, Hội nghị Internet VNNIC 2026.

5 Tốc độ: bit và byte

Nhà mạng ghi Mbps, phần mềm tải file thường hiện MB/s. 1 byte bằng 8 bit, nên hai con số chênh nhau 8 lần.

```
file_mb = 700                 # dung lượng file, megabyte
speed_mbps = 100              # tốc độ mạng, megabit mỗi giây

seconds = file_mb * 8 / speed_mbps
print(seconds, "giây")
print(speed_mbps / 8, "MB/s")
```

KẾT QUẢ KHI CHẠY

```
56.0 giây
12.5 MB/s
```

Thuật ngữ	Ý nghĩa
Băng thông	Tốc độ tối đa của đường truyền
Thông lượng	Tốc độ thực tế đo được
Độ trễ	Thời gian dữ liệu đi từ nguồn tới đích
Jitter	Độ dao động của độ trễ
Mất gói	Tỷ lệ gói không tới đích

Trong truyền thông, kilo và mega thường tính theo cơ số 10: 1 Mbps là 1.000.000 bit/s.

6 Bốn loại trễ

Tổng trễ tại mỗi nút = xử lý + hàng đợi + truyền + lan truyền. Bài thi hay nhầm giữa truyền và lan truyền.

```
L = 1500 * 8        # kích thước gói, bit
R = 100e6           # tốc độ đường truyền, bit/s
d = 1000e3          # khoảng cách, mét
s = 2e8             # tốc độ tín hiệu trong cáp, m/s

d_trans = L / R     # đẩy hết bit lên đường truyền
d_prop = d / s      # một bit đi hết quãng đường

print(f"Truyền:      {d_trans * 1000:.3f} ms")
print(f"Lan truyền: {d_prop * 1000:.3f} ms")
print(f"Tổng:       {(d_trans + d_prop) * 1000:.3f} ms")
```

KẾT QUẢ KHI CHẠY

```
Truyền:      0.120 ms
Lan truyền: 5.000 ms
Tổng:       5.120 ms
```

Loại trễ	Phụ thuộc vào
Xử lý	Thời gian router đọc header, kiểm tra lỗi
Hàng đợi	Mức độ đông của mạng, thay đổi liên tục
Truyền	Kích thước gói L và tốc độ R, bằng L / R
Lan truyền	Khoảng cách d và tốc độ tín hiệu s, bằng d / s

Mẹo nhớ: truyền là thời gian xe qua trạm thu phí, lan truyền là thời gian xe chạy trên đường.

7 Vì sao phải chia gói?

Router nhận đủ một gói rồi mới chuyển tiếp. Chia nhỏ giúp các chặng làm việc song song như dây chuyền.

```
def send_time(total, packet, rate, links):
    n = total // packet           # số gói
    t = packet / rate             # thời gian đẩy một gói
    return (n + links - 1) * t    # gói cuối ra khỏi chặng cuối

msg = 8_000_000                   # 1 MB tính bằng bit
print(send_time(msg, msg, 1e6, 3), "giây: gửi nguyên khối")
print(send_time(msg, 8000, 1e6, 3), "giây: chia gói 1 KB")
```

KẾT QUẢ KHI CHẠY

```
24.0 giây: gửi nguyên khối
8.016 giây: chia gói 1 KB
```

- Công thức bài thi: thời gian = (N + M − 1) · L / R, với N gói và M chặng.
- Gói nhỏ giúp các chặng truyền song song, và mất một gói chỉ phải gửi lại gói đó.
- Gói quá nhỏ lại tốn header, nên phải cân bằng.
- Công thức này bỏ qua trễ lan truyền, xử lý và hàng đợi.

8 Vì sao mạng chia tầng?

Mỗi tầng giải quyết một việc và chỉ làm việc với tầng kề nó. Thay công nghệ ở một tầng không buộc phải sửa các tầng khác.

VÍ DỤ GỬI THƯ

Bạn viết thư. Bưu điện lo phân loại. Xe tải lo chở. Bạn không cần biết xe chạy đường nào.

VÍ DỤ MẠNG

Trình duyệt dùng HTTP. Đổi từ Wi-Fi sang 4G, trình duyệt vẫn chạy mà không phải sửa code.

Khái niệm	Ý nghĩa
Dịch vụ	Tầng dưới cung cấp cho tầng trên
Giao thức	Quy tắc giữa hai tầng cùng cấp ở hai máy
Giao diện	Điểm tầng trên gọi tầng dưới
PDU	Đơn vị dữ liệu của mỗi tầng

Nhược điểm: thêm header và thêm bước xử lý ở mỗi tầng.

9 Mô hình OSI 7 tầng

Tầng	Tên	Nhiệm vụ	Ví dụ
7	Ứng dụng	Dịch vụ cho người dùng	HTTP, DNS, SMTP
6	Trình diễn	Định dạng, mã hóa, nén	JPEG, TLS, UTF-8
5	Phiên	Thiết lập, duy trì phiên	RPC, NetBIOS
4	Giao vận	Truyền giữa các tiến trình	TCP, UDP
3	Mạng	Địa chỉ logic, chọn đường	IP, ICMP, OSPF
2	Liên kết dữ liệu	Truyền qua một chặng, phát hiện lỗi	Ethernet, Wi-Fi
1	Vật lý	Truyền bit qua môi trường	Cáp, sóng, tín hiệu

Mẹo nhớ từ tầng 7 xuống tầng 1: All People Seem To Need Data Processing.

THIẾT BỊ THEO TẦNG
Tầng 1: hub, repeater. Tầng 2: switch, bridge. Tầng 3: router.

LƯU Ý
OSI là mô hình tham chiếu để học và phân tích. Internet thực tế chạy theo TCP/IP.

Vị trí của TLS có thể được xếp khác nhau tùy tài liệu, vì giao thức thật không khớp hoàn toàn với OSI.

10 Mô hình TCP/IP và so sánh

OSI	TCP/IP	PDU
Ứng dụng, Trình diễn, Phiên	Ứng dụng	Dữ liệu
Giao vận	Giao vận	Đoạn (TCP), datagram (UDP)
Mạng	Internet	Gói tin
Liên kết dữ liệu	Truy nhập mạng	Khung
Vật lý	Truy nhập mạng	Bit

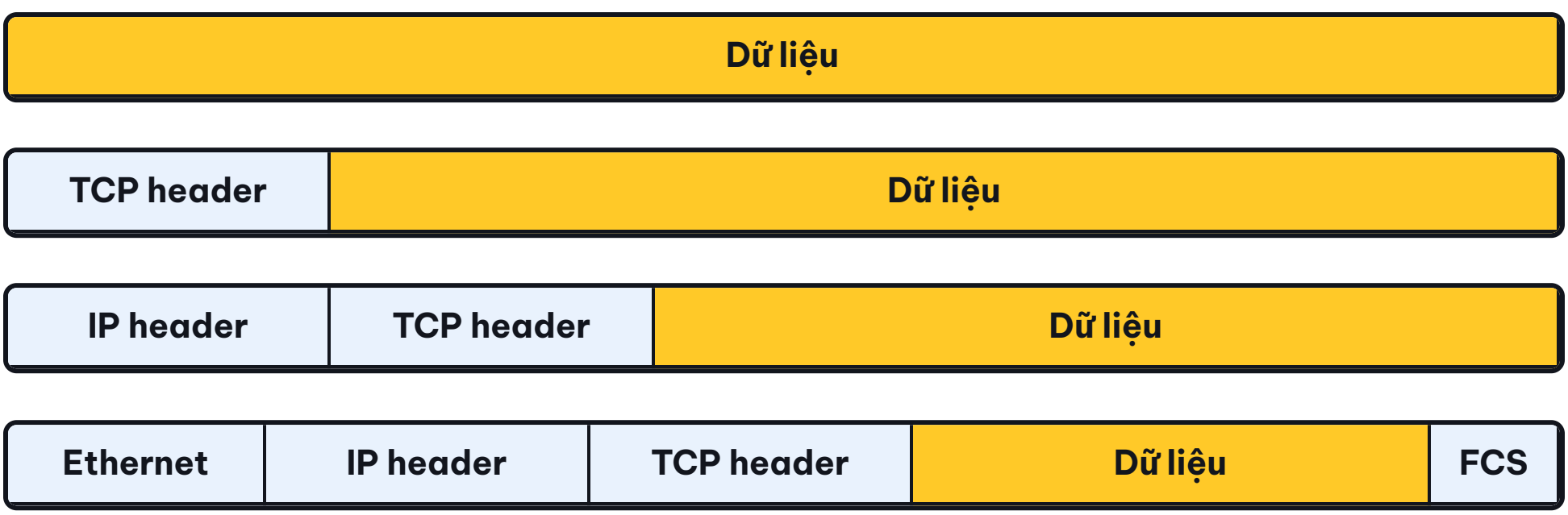
OSI
Ra đời như một chuẩn tham chiếu, phân định rõ dịch vụ, giao diện, giao thức.

TCP/IP
Xuất phát từ giao thức thực tế, là nền của Internet.

Nhiều giáo trình dùng mô hình 5 tầng: Ứng dụng, Giao vận, Mạng, Liên kết, Vật lý. Hãy trả lời theo mô hình mà đề yêu cầu.

11 Đóng gói dữ liệu

Đi xuống mỗi tầng, dữ liệu được thêm header. Bên nhận đi ngược lên và bóc từng lớp.




```
data = b"Xin chao"
tcp = b"[TCP]" + data
ip = b"[IP]" + tcp
frame = b"[ETH]" + ip + b"[FCS]"
print(frame)

received = frame.removeprefix(b"[ETH]").removesuffix(b"[FCS]")
received = received.removeprefix(b"[IP]").removeprefix(b"[TCP]")
print(received)
```

KẾT QUẢ KHI CHẠY

```
b'[ETH][IP][TCP]Xin chao[FCS]'
b'Xin chao'
```

- ✓ Router chỉ cần bóc tới tầng 3 để biết chuyển gói đi đâu, rồi đóng khung mới cho chặng sau.
- ✓ Địa chỉ IP thường giữ nguyên suốt đường. Địa chỉ MAC đổi theo từng chặng.

Một gói tin đi qua mạng

Điều gì xảy ra khi bạn gõ một địa chỉ web và nhấn Enter?

1. **Hỏi DNS**
   Trình duyệt cần địa chỉ IP của tên miền.

2. **So địa chỉ, gọi ARP**
   Đích khác mạng thì gửi tới gateway, dùng ARP để biết MAC của gateway.

3. **Bắt tay TCP**
   Máy và server thiết lập kết nối, với HTTPS còn thêm bắt tay TLS.

4. **Gửi yêu cầu HTTP**
   Dữ liệu được đóng gói và đi qua switch, router, nhà mạng.

5. **Router chuyển tiếp**
   Mỗi router tra bảng định tuyến, giảm TTL, đóng khung mới.

6. **Nhận phản hồi**
   Server trả dữ liệu, trình duyệt hiển thị trang.

Đây là câu phỏng vấn kinh điển. Sau cuốn này, bạn nên kể được từng bước với giao thức tương ứng.

12 Tín hiệu và giới hạn tốc độ

Nyquist cho kênh không nhiễu, Shannon cho kênh có nhiễu. Tốc độ thật không thể vượt giới hạn Shannon.

```python
from math import log2

B = 3000                    # băng thông 3 kHz
L = 4                       # 4 mức tín hiệu
print("Nyquist:", 2 * B * log2(L), "bit/s")

snr_db = 30                 # tỉ số tín hiệu trên nhiễu
snr = 10 ** (snr_db / 10)
print("Shannon:", round(B * log2(1 + snr)), "bit/s")
```

KẾT QUẢ KHI CHẠY

```
Nyquist: 12000.0 bit/s
Shannon: 29902 bit/s
```

Công thức	Ý nghĩa
$C = 2B \log_2 L$	Nyquist, L là số mức tín hiệu
$C = B \log_2(1 + S/N)$	Shannon, S/N là tỉ số công suất
$dB = 10 \log_{10}(S/N)$	Đổi sang đơn vị decibel

LỖI HAY GẶP

Dùng thẳng số dB trong Shannon. Phải đổi ra tỉ số trước.

BAUD VÀ BIT

Baud là số lần đổi tín hiệu mỗi giây. Một lần đổi có thể mang nhiều bit.

13 Mã hóa và điều chế

Mã hóa đường truyền biến bit thành tín hiệu số. Điều chế gắn dữ liệu vào sóng mang.

```
def nrz(bits):
    return ["+" if b == "1" else "-" for b in bits]

def manchester(bits):    # theo IEEE 802.3
    return ["-+" if b == "1" else "+-" for b in bits]

print(nrz("1011"))
print(manchester("1011"))
```

KẾT QUẢ KHI CHẠY

```
['+', '-', '+', '+']
['-+', '+-', '-+', '-+']
```

Tên	Thay đổi	Ghi chú
Manchester	Chuyển mức giữa mỗi bit	Tự đồng bộ, tốn gấp đôi tần số
ASK	Biên độ	Đơn giản, dễ nhiễu
FSK	Tần số	Chống nhiễu tốt hơn ASK
PSK	Pha	Dùng nhiều trong không dây
QAM	Biên độ và pha	Nhiều bit mỗi ký hiệu

Có tài liệu dùng quy ước Manchester ngược lại, nên hãy xem đề quy định thế nào. QAM-64 mang $\log_2 64 = 6$ bit mỗi ký hiệu.

14 Dồn kênh

Nhiều luồng dữ liệu dùng chung một đường truyền.

Kỹ thuật	Chia theo	Ví dụ
FDM	Tần số	Phát thanh FM
TDM	Khe thời gian	Mạng điện thoại số
WDM	Bước sóng ánh sáng	Đường trục cáp quang
CDM	Mã	CDMA trong 3G
OFDM	Nhiều sóng mang con	Wi-Fi, 4G, 5G

Ví dụ dễ nhớ: FDM là mỗi người hát một tông, TDM là mỗi người nói một lượt, CDM là mỗi cặp nói một ngôn ngữ.

15 Môi trường truyền dẫn

Cáp đồng xoắn đôi	Tốc độ thường gặp	Khoảng cách
Cat5e	1 Gbps	100 m
Cat6	1 Gbps, 10 Gbps đoạn ngắn	100 m, 55 m với 10 Gbps
Cat6a	10 Gbps	100 m

Cáp quang	Đặc điểm	Dùng khi
Đơn mode (SMF)	Lõi nhỏ, đi rất xa	Đường trục, nhà mạng
Đa mode (MMF)	Lõi lớn, cự ly ngắn hơn	Trong tòa nhà, trung tâm dữ liệu

CÁP ĐỒNG
Rẻ, dễ thi công. Chịu nhiễu điện từ, giới hạn 100 m mỗi đoạn.

CÁP QUANG
Nhanh, xa, không nhiễu điện từ. Đắt hơn, khó hàn nối hơn.

KHÔNG DÂY
Tiện di chuyển. Dùng chung môi trường, dễ nhiễu và cần bảo mật tốt.

UTP VÀ STP
UTP không có lớp bọc chống nhiễu. STP có, dùng nơi nhiều nhiễu.

Số liệu là giá trị thường dùng theo chuẩn cáp. Tốc độ thực tế còn phụ thuộc thiết bị và chất lượng thi công.

16 Bấm cáp mạng

Chân	T568A	T568B
1	Trắng xanh lá	Trắng cam
2	Xanh lá	Cam
3	Trắng cam	Trắng xanh lá
4	Xanh dương	Xanh dương
5	Trắng xanh dương	Trắng xanh dương
6	Cam	Xanh lá
7	Trắng nâu	Trắng nâu
8	Nâu	Nâu

CÁP THẲNG
Hai đầu cùng chuẩn. Nối máy với switch, router với switch.

CÁP CHÉO
Một đầu A, một đầu B. Nối hai thiết bị cùng loại.

Thiết bị hiện đại có Auto-MDIX tự đảo chân, nhưng bài thi và Packet Tracer vẫn hay hỏi loại cáp.

17 Thiết bị mạng theo tầng

Thiết bị	Tầng	Nhiệm vụ
Repeater, Hub	1	Khuếch đại và phát tín hiệu ra các cổng
Bridge, Switch	2	Chuyển khung theo địa chỉ MAC
Router	3	Chuyển gói giữa các mạng theo địa chỉ IP
Switch tầng 3	2, 3	Chuyển mạch và định tuyến giữa VLAN
Access Point	2	Nối thiết bị không dây vào mạng có dây
Firewall	3 trở lên	Lọc lưu lượng theo chính sách

MIỀN XUNG ĐỘT
Mỗi cổng switch là một miền xung đột riêng. Hub gộp tất cả làm một.

MIỀN QUẢNG BÁ
Router chia miền quảng bá. Switch không chia, trừ khi dùng VLAN.

"Modem Wi-Fi" nhà mạng lắp thường gộp modem quang, router, switch và access point trong một hộp.

18 Nhiệm vụ của tầng liên kết

Tầng này đưa khung đi qua một chặng: từ máy tới switch, hoặc từ router này sang router kế tiếp.

Nhiệm vụ	Cách làm
Đóng khung	Thêm header và trailer quanh gói tin
Định địa chỉ	Dùng địa chỉ MAC trong cùng một mạng
Phát hiện lỗi	Parity, checksum, CRC
Truy nhập kênh	Quyết định ai được truyền trên đường chung
Truyền tin cậy	Một số công nghệ như Wi-Fi có xác nhận từng khung

LLC
Lớp con trên, giao tiếp với tầng mạng.

MAC
Lớp con dưới, lo địa chỉ và truy nhập đường truyền.

Thiết bị thường làm phần lớn công việc này trên card mạng, nên nó nằm giữa phần cứng và phần mềm.

19 Parity và checksum

Thêm vài bit kiểm tra để bên nhận biết dữ liệu có bị đổi trên đường hay không.

```
def even_parity(bits):
    return bits.count("1") % 2

def checksum(data):                      # Internet checksum
    s = 0
    for i in range(0, len(data), 2):
        s += data[i] << 8 | data[i + 1]
        s = (s & 0xFFFF) + (s >> 16)  # cộng phần nhớ vòng lại
    return ~s & 0xFFFF

print(even_parity("1011001"))

d = bytes([0x45, 0x00, 0x00, 0x1C])
c = checksum(d)
print(hex(c))
print(checksum(d + c.to_bytes(2, "big")))   # 0: không lỗi
```

KẾT QUẢ KHI CHẠY

```
0
0xbae3
0
```

Cách	Khả năng	Dùng ở
Parity	Phát hiện số lẻ bit lỗi	Bộ nhớ, đường truyền đơn giản
Checksum	Cài đặt nhanh, yếu hơn CRC	IP, TCP, UDP
CRC	Rất tốt với lỗi cụm	Ethernet, Wi-Fi

20 CRC tính bằng tay

Thêm r bit 0 vào sau dữ liệu, rồi chia cho đa thức sinh bằng phép XOR. Phần dư chính là CRC.

```python
def crc(data, poly):
    bits = list(data + "0" * (len(poly) - 1))
    for i in range(len(data)):
        if bits[i] == "1":
            for j, p in enumerate(poly):
                bits[i + j] = str(int(bits[i + j]) ^ int(p))
    return "".join(bits[len(data):])

G = "1001"                              # đa thức sinh x³ + 1
r = crc("101110", G)
print("CRC:", r)
print("Gửi:", "101110" + r)
print("Bên nhận:", crc("101110" + r, G))    # dư 0
print("Khung lỗi:", crc("101100" + r, G))   # dư khác 0
```

KẾT QUẢ KHI CHẠY

```
CRC: 011
Gửi: 101110011
Bên nhận: 000
Khung lỗi: 010
```

- ✓ Đa thức bậc r có r + 1 bit, và sinh ra r bit CRC.
- ✓ Phép chia dùng XOR, không có nhớ và mượn.
- ✓ Ethernet dùng CRC-32 trong trường FCS.
- ✓ Bên nhận chỉ phát hiện lỗi rồi bỏ khung, không tự sửa được.

21 Mã Hamming sửa lỗi

Đặt bit kiểm tra ở vị trí 1, 2, 4. Tổ hợp các kết quả kiểm tra chỉ thẳng vào vị trí bit sai.

```
def encode(d):                          # 4 bit dữ liệu thành 7 bit
    d1, d2, d3, d4 = map(int, d)
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p3 = d2 ^ d3 ^ d4
    return [p1, p2, d1, p3, d2, d3, d4]

def fix(c):
    s1 = c[0] ^ c[2] ^ c[4] ^ c[6]
    s2 = c[1] ^ c[2] ^ c[5] ^ c[6]
    s3 = c[3] ^ c[4] ^ c[5] ^ c[6]
    pos = s3 * 4 + s2 * 2 + s1    # vị trí bit sai
    if pos:
        c[pos - 1] ^= 1
    return pos, c

code = encode("1011")
print(code)
code[4] ^= 1                            # làm hỏng bit thứ 5
print(fix(code))
```

KẾT QUẢ KHI CHẠY

```
[0, 1, 1, 0, 0, 1, 1]
(5, [0, 1, 1, 0, 0, 1, 1])
```

HAMMING (7,4)
Sửa được 1 bit sai trong mỗi 7 bit.

SỐ BIT KIỂM TRA
Cần r sao cho 2^r ≥ m + r + 1, với m bit dữ liệu.

22 ALOHA và hiệu suất

Nhiều máy dùng chung một kênh thì cần luật để hạn chế xung đột.

```
from math import e

def slotted(n):                 # hiệu suất với p = 1 / n
    p = 1 / n
    return n * p * (1 - p) ** (n - 1)

for n in (2, 10, 100):
    print(n, "máy:", round(slotted(n), 3))
print("Giới hạn slotted:", round(1 / e, 3))
print("Giới hạn pure:", round(1 / (2 * e), 3))
```

KẾT QUẢ KHI CHẠY

```
2 máy: 0.5
10 máy: 0.387
100 máy: 0.37
Giới hạn slotted: 0.368
Giới hạn pure: 0.184
```

Nhóm	Cách làm	Ví dụ
Chia kênh	Mỗi máy một phần cố định	TDMA, FDMA
Truy nhập ngẫu nhiên	Cứ gửi, xung đột thì thử lại	ALOHA, CSMA
Lần lượt	Chờ đến lượt	Polling, Token

Đề thi hay hỏi hai con số: Pure ALOHA khoảng 18,4% và Slotted ALOHA khoảng 36,8%.

23 CSMA/CD và CSMA/CA

Nghe kênh trước khi gửi. Ethernet kiểu cũ phát hiện xung đột, còn Wi-Fi tìm cách tránh xung đột.

```
rate = 10e6
print(round(512 / rate * 1e6, 1), "µs mỗi khe ở 10 Mbps")

for n in range(1, 5):                    # backoff lũy thừa nhị phân
    print(f"Lần {n}: chọn K từ 0 đến {2 ** min(n, 10) - 1}")

d, s = 2500, 2e8          # chiều dài cáp, tốc độ tín hiệu
print("Khung tối thiểu:", 2 * d / s * rate, "bit")
```

KẾT QUẢ KHI CHẠY

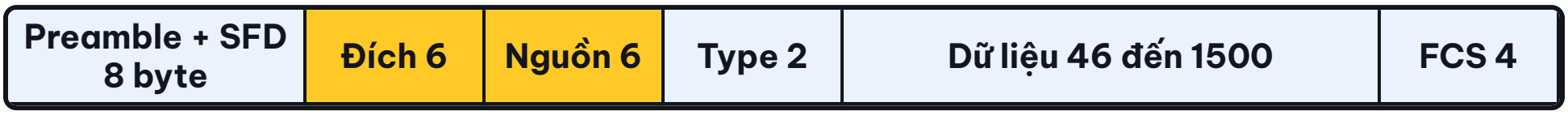
```
51.2 µs mỗi khe ở 10 Mbps
Lần 1: chọn K từ 0 đến 1
Lần 2: chọn K từ 0 đến 3
Lần 3: chọn K từ 0 đến 7
Lần 4: chọn K từ 0 đến 15
Khung tối thiểu: 250.0 bit
```

	CSMA/CD	CSMA/CA
Dùng ở	Ethernet dùng chung, bán song công	Wi-Fi
Xung đột	Phát hiện khi đang gửi	Tránh trước, chờ ACK sau
Khi lỗi	Dừng, gửi jam, chờ ngẫu nhiên	Không có ACK thì gửi lại

- Công thức: L ≥ 2 · d · R / s. Khung phải đủ dài để còn đang gửi khi tín hiệu xung đột quay về.
- Ethernet song công qua switch ngày nay không còn xung đột.

24 Địa chỉ MAC và khung Ethernet

MAC dài 48 bit, 24 bit đầu thường là mã nhà sản xuất. Địa chỉ quảng bá là ff:ff:ff:ff:ff:ff.




```python
def info(mac):
    first = int(mac.split(":")[0], 16)
    return mac[:8], bool(first & 1), bool(first & 2)

print(info("3c:22:fb:12:34:56"))     # OUI, multicast, local
print(info("ff:ff:ff:ff:ff:ff"))

print("Tối thiểu:", 6 + 6 + 2 + 46 + 4, "byte")
print("Tối đa:", 6 + 6 + 2 + 1500 + 4, "byte")
```

KẾT QUẢ KHI CHẠY

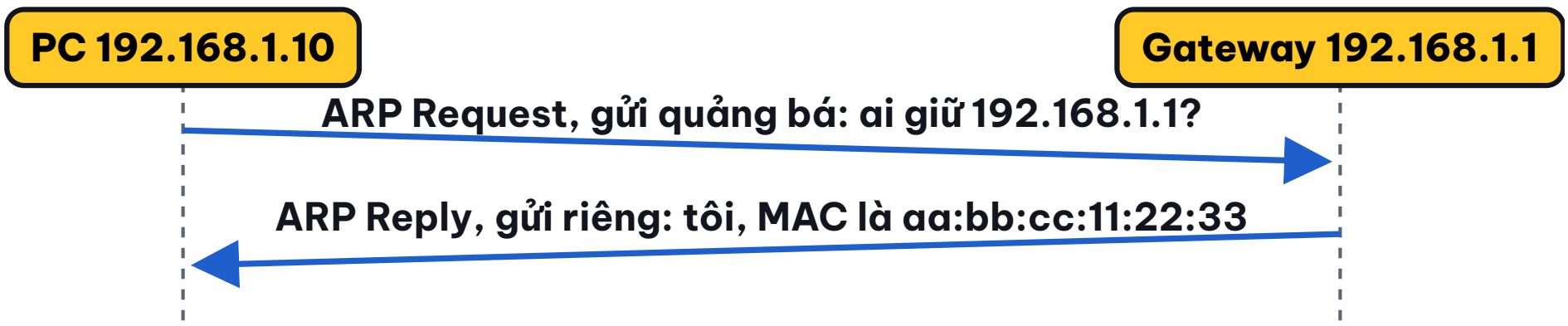
```
('3c:22:fb', False, False)
('ff:ff:ff', True, True)
Tối thiểu: 64 byte
Tối đa: 1518 byte
```

Giá trị Type	Giao thức bên trong
`0x0800`	IPv4
`0x0806`	ARP
`0x86DD`	IPv6
`0x8100`	Khung có thẻ VLAN 802.1Q

Dữ liệu nhỏ hơn 46 byte sẽ được đệm thêm. Kích thước 64 và 1518 byte không tính Preamble.

25 ARP: từ IP ra MAC

Biết IP đích nhưng chưa biết MAC để đóng khung, máy phát một câu hỏi quảng bá trong mạng.



Tình huống	Máy hỏi MAC của ai?
Đích cùng mạng	Chính máy đích
Đích khác mạng	Gateway mặc định

- Kết quả được lưu trong bảng ARP một thời gian ngắn. Xem bằng lệnh `arp -a`.
- ARP không xác thực, nên có thể bị giả mạo. Đây là nền của tấn công ARP spoofing.
- IPv6 không dùng ARP mà dùng NDP.

Câu hay hỏi: ARP Request là quảng bá, ARP Reply là gửi riêng.

26 Switch học địa chỉ

Switch ghi nhớ MAC nguồn ở cổng nào. Chưa biết đích thì gửi ra mọi cổng khác.

```
class Switch:
    def __init__(self, ports):
        self.ports, self.table = ports, {}

    def receive(self, src, dst, port):
        self.table[src] = port                  # học MAC nguồn
        if dst in self.table:
            out = self.table[dst]
            return [] if out == port else [out]
        return [p for p in self.ports if p != port]   # flood

sw = Switch([1, 2, 3, 4])
print(sw.receive("A", "B", 1))   # chưa biết B: gửi hết
print(sw.receive("B", "A", 2))   # đã biết A: gửi riêng
print(sw.receive("A", "B", 1))   # đã biết B: gửi riêng
print(sw.table)
```

KẾT QUẢ KHI CHẠY

```
[2, 3, 4]
[1]
[2]
{'A': 1, 'B': 2}
```

Hành động	Khi nào
Flood	Chưa biết đích, hoặc khung quảng bá
Forward	Biết đích, nằm ở cổng khác
Filter	Biết đích, nằm ngay cổng vừa nhận

Mục trong bảng MAC sẽ hết hạn nếu không thấy lưu lượng. Xem bằng lệnh `show mac address-table`.

27 VLAN

Chia một switch vật lý thành nhiều mạng logic. Các VLAN khác nhau không gặp nhau nếu không qua thiết bị tầng 3.

Đích	Nguồn	Thẻ 802.1Q 4 byte	Type	Dữ liệu	FCS

```python
import struct

def dot1q_tag(vlan, priority=0):
    return struct.pack("!HH", 0x8100, priority << 13 | vlan)

print(dot1q_tag(20).hex())
print("VLAN dùng được:", 2 ** 12 - 2)    # bỏ VLAN 0 và 4095
```

KẾT QUẢ KHI CHẠY

```
81000014
VLAN dùng được: 4094
```

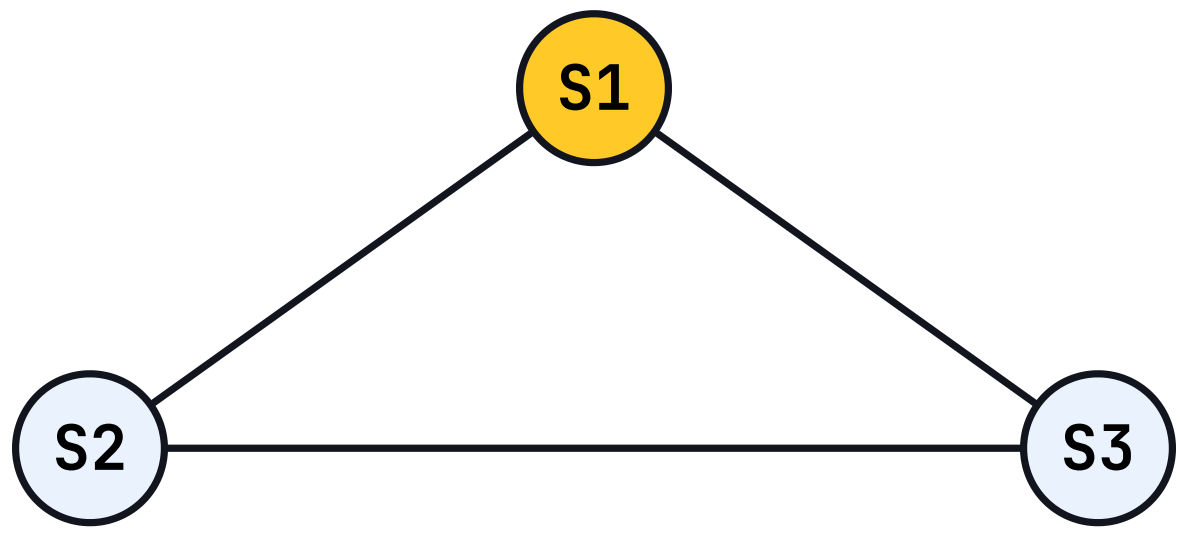
```
Switch(config)# vlan 10
Switch(config-vlan)# name SinhVien
Switch(config)# interface fa0/1
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 10
Switch(config)# interface gi0/1
Switch(config-if)# switchport mode trunk
Switch# show vlan brief
```

CỔNG ACCESS
Thuộc một VLAN, nối tới thiết bị đầu cuối.

CỔNG TRUNK
Mang nhiều VLAN, nối giữa các switch. VLAN native không gắn thẻ.

28 STP và EtherChannel

Nối vòng giữa các switch để dự phòng sẽ gây bão quảng bá, vì khung Ethernet không có TTL. STP chặn bớt một cổng để phá vòng.



S1 là root bridge. Một cổng trên đường S2 và S3 bị chặn, sẽ mở lại khi có đường hỏng.

Bước	Quy tắc
1. Chọn root	Bridge ID nhỏ nhất: ưu tiên rồi tới MAC
2. Root port	Mỗi switch khác chọn cổng tới root rẻ nhất
3. Designated port	Mỗi đoạn mạng chọn một cổng chuyển tiếp
4. Cổng còn lại	Bị chặn

RSTP (802.1w)
Hội tụ nhanh hơn nhiều so với STP gốc 802.1D.

ETHERCHANNEL
Gộp nhiều cáp thành một đường logic, tăng băng thông và dự phòng. LACP là chuẩn mở.

29 Wi-Fi

Tên	Chuẩn	Băng tần
Wi-Fi 4	802.11n	2,4 và 5 GHz
Wi-Fi 5	802.11ac	5 GHz
Wi-Fi 6	802.11ax	2,4 và 5 GHz
Wi-Fi 6E	802.11ax	Thêm 6 GHz
Wi-Fi 7	802.11be	2,4, 5 và 6 GHz

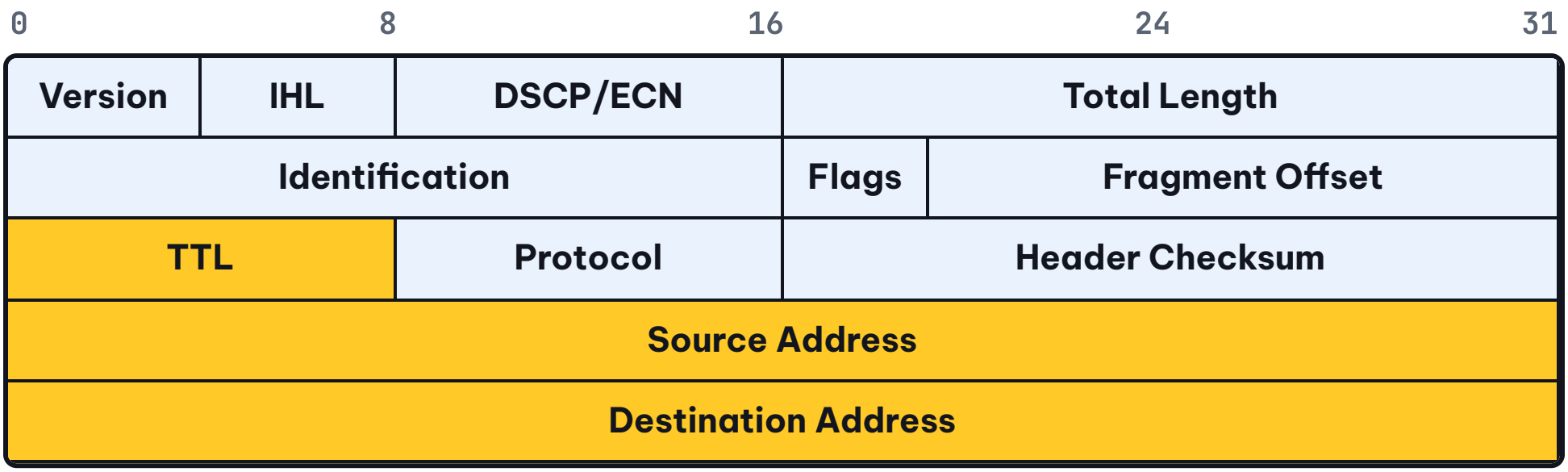


Trạm ẩn: A và B cùng nghe được AP, nhưng không nghe được nhau.

- Trạm ẩn khiến nghe kênh không đủ. RTS/CTS giúp đặt trước đường truyền.
- Băng 2,4 GHz đi xa hơn nhưng đông. Ba kênh không chồng lấn thường dùng là 1, 6, 11.
- Băng 5 GHz nhanh và thoáng hơn, nhưng xuyên tường kém hơn.
- SSID là tên mạng. BSSID là MAC của access point.

Khung Wi-Fi có thể dùng tới 4 địa chỉ MAC, vì phải ghi thêm access point trên đường đi.

30 Header IPv4




```python
import socket, struct

raw = bytes.fromhex("45000028abcd400040060000c0a8010508080808")
(vi, tos, total, ident, flags, ttl, proto, csum, src, dst
 ) = struct.unpack("!BBHHHBBH4s4s", raw)

print("Version", vi >> 4, "| Header", (vi & 15) * 4, "byte")
print("TTL", ttl, "| Protocol", proto, "| DF", bool(flags & 0x4000))
print(socket.inet_ntoa(src), "->", socket.inet_ntoa(dst))
```

KẾT QUẢ KHI CHẠY

```
Version 4 | Header 20 byte
TTL 64 | Protocol 6 | DF True
192.168.1.5 -> 8.8.8.8
```

Trường	Ý nghĩa
IHL	Độ dài header, đơn vị 4 byte, tối thiểu 5 tức 20 byte
TTL	Giảm 1 sau mỗi router, về 0 thì gói bị hủy
Protocol	Tầng trên: ICMP 1, TCP 6, UDP 17
Identification, Flags, Offset	Dùng khi phân mảnh
Header Checksum	Chỉ kiểm tra header, tính lại ở mỗi router

31 Địa chỉ IPv4

IPv4 dài 32 bit, viết thành 4 số thập phân. Cách chia lớp nay chỉ còn ý nghĩa lịch sử, nhưng vẫn rất hay ra đề.

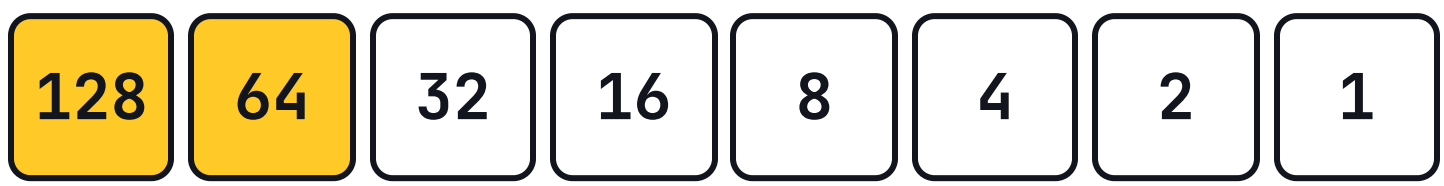
Lớp	Octet đầu	Bit đầu	Mặt nạ mặc định	Dùng cho
A	1 đến 126	0	/8	Mạng rất lớn
B	128 đến 191	10	/16	Mạng vừa
C	192 đến 223	110	/24	Mạng nhỏ
D	224 đến 239	1110	Không có	Multicast
E	240 đến 255	1111	Không có	Dự trữ

Dải đặc biệt	Ý nghĩa
10.0.0.0/8	Riêng (RFC 1918)
172.16.0.0/12	Riêng, từ 172.16 đến 172.31
192.168.0.0/16	Riêng, hay dùng ở gia đình
127.0.0.0/8	Loopback, chính máy mình
169.254.0.0/16	Tự gán khi không nhận được DHCP
100.64.0.0/10	Dùng cho NAT cấp nhà mạng

Dải riêng không đi trực tiếp ra Internet mà phải qua NAT.

32 Nhị phân cho subnet

Chỉ cần thuộc 8 giá trị của một octet là tính subnet nhanh. Không cần máy tính.




```
def to_bin(ip):
    return ".".join(f"{int(x):08b}" for x in ip.split("."))

print(to_bin("192.168.1.130"))
print(to_bin("255.255.255.192"))
print(int("11000000", 2), int("10000010", 2))
```

KẾT QUẢ KHI CHẠY

```
11000000.10101000.00000001.10000010
11111111.11111111.11111111.11000000
192 130
```

Số bit 1	1	2	3	4	5	6	7	8
Octet mặt nạ	128	192	224	240	248	252	254	255

Ví dụ: /26 có 2 bit 1 ở octet cuối, nên octet đó là 192. Mặt nạ là 255.255.255.192.

33 Subnet mask và CIDR

Mặt nạ tách địa chỉ thành phần mạng và phần máy. AND với mặt nạ cho ra địa chỉ mạng.

```
def to_int(ip):
    a, b, c, d = map(int, ip.split("."))
    return a << 24 | b << 16 | c << 8 | d

def to_ip(n):
    return ".".join(str(n >> s & 255) for s in (24, 16, 8, 0))

def subnet(ip, prefix):
    mask = 0xFFFFFFFF << (32 - prefix) & 0xFFFFFFFF
    net = to_int(ip) & mask
    broadcast = net | ~mask & 0xFFFFFFFF
    hosts = 2 ** (32 - prefix) - 2
    return to_ip(mask), to_ip(net), to_ip(broadcast), hosts

print(subnet("192.168.1.130", 26))
print(subnet("172.16.45.14", 20))
```

KẾT QUẢ KHI CHẠY

```
('255.255.255.192', '192.168.1.128', '192.168.1.191', 62)
('255.255.240.0', '172.16.32.0', '172.16.47.255', 4094)
```

Khái niệm	Quy tắc
Địa chỉ mạng	Mọi bit phần máy bằng 0
Địa chỉ quảng bá	Mọi bit phần máy bằng 1
Số máy dùng được	2^h − 2, với h bit phần máy
Ngoại lệ	/31 dùng cho liên kết điểm-điểm, /32 chỉ một máy

34 Tính subnet nhanh bằng tay

Đề: tìm thông tin của 192.168.1.130/26. Dùng cách bước nhảy, xong trong khoảng 30 giây.

1 Tìm octet thú vị
/26 nằm ở octet thứ 4, mặt nạ là 255.255.255.192.

2 Tính bước nhảy
256 − 192 = 64. Các mạng bắt đầu ở 0, 64, 128, 192.

3 Tìm mạng chứa IP
130 nằm giữa 128 và 192, nên địa chỉ mạng là 192.168.1.128.

4 Tìm quảng bá và dải máy
Quảng bá là .191. Máy dùng được từ .129 đến .190.

Lời giải câu đố ở trang bìa: mạng 192.168.1.64/26 chỉ gồm .64 đến .127, nên .130 nằm ngoài và phải đi qua router.

```
import ipaddress as ip
n = ip.ip_interface("192.168.1.130/26").network
print(n, n.broadcast_address, n[1], n[-2])
```

KẾT QUẢ KHI CHẠY

```
192.168.1.128/26 192.168.1.191 192.168.1.129 192.168.1.190
```

Bảng subnet nên thuộc

```python
import ipaddress as ip

print("CIDR  Mặt nạ            Bước  Máy")
for p in range(24, 31):
    n = ip.ip_network(f"0.0.0.0/{p}")
    block = 2 ** (32 - p)
    print(f"/{p:<4} {str(n.netmask):<18} {block:<5} {block - 2}")
```

KẾT QUẢ KHI CHẠY

```
CIDR  Mặt nạ            Bước  Máy
/24   255.255.255.0       256   254
/25   255.255.255.128     128   126
/26   255.255.255.192     64    62
/27   255.255.255.224     32    30
/28   255.255.255.240     16    14
/29   255.255.255.248     8     6
/30   255.255.255.252     4     2
```

Cần bao nhiêu máy?	Chọn
2 máy, nối router	/30
Tới 14 máy	/28
Tới 30 máy	/27
Tới 62 máy	/26
Tới 126 máy	/25
Tới 254 máy	/24

Với /16 đến /23, cách tính giống hệt, chỉ là octet thú vị chuyển sang octet thứ 3.

35 Chia đều mạng con

Đề: chia 192.168.10.0/24 thành 4 mạng con bằng nhau. Mượn n bit thì được 2^n mạng.

```
import ipaddress as ip

def end(a):
    return "." + str(a).split(".")[-1]

net = ip.ip_network("192.168.10.0/24")
for s in net.subnets(new_prefix=26):
    print(s, "máy", end(s[1]), "-", end(s[-2]), "quảng bá", end(s[-1]))
```

KẾT QUẢ KHI CHẠY

```
192.168.10.0/26 máy .1 - .62 quảng bá .63
192.168.10.64/26 máy .65 - .126 quảng bá .127
192.168.10.128/26 máy .129 - .190 quảng bá .191
192.168.10.192/26 máy .193 - .254 quảng bá .255
```

1 Tính số bit mượn
2^n ≥ 4 nên n = 2. Mặt nạ mới là /24 + 2 = /26.

2 Kiểm tra số máy
Còn 6 bit phần máy, được $2^6 - 2 = 62$ máy mỗi mạng.

3 Liệt kê theo bước nhảy
Bước nhảy 64: các mạng .0, .64, .128, .192.

Đề cho theo số máy: chọn h nhỏ nhất với 2^h – 2 ≥ số máy, rồi /prefix = 32 – h.

36 VLSM: chia mạng theo nhu cầu

Mỗi mạng dùng một cỡ khác nhau để đỡ lãng phí. Luôn cấp cho mạng lớn nhất trước.

```
import ipaddress as ip
from math import ceil, log2

def vlsm(base, needs):
    addr = int(ip.ip_network(base).network_address)
    out = []
    for name, hosts in sorted(needs.items(), key=lambda x: -x[1]):
        bits = ceil(log2(hosts + 2))     # cộng địa chỉ mạng, quảng bá
        net = ip.ip_network((addr, 32 - bits))
        out.append((name, hosts, str(net)))
        addr += 2 ** bits
    return out

needs = {"Phòng máy": 100, "Giảng viên": 50,
         "Thư viện": 20, "Nối router": 2}
for row in vlsm("192.168.1.0/24", needs):
    print(row)
```

KẾT QUẢ KHI CHẠY

```
('Phòng máy', 100, '192.168.1.0/25')
('Giảng viên', 50, '192.168.1.128/26')
('Thư viện', 20, '192.168.1.192/27')
('Nối router', 2, '192.168.1.224/30')
```

- ✓ Cỡ khối là lũy thừa của 2, đủ cho số máy cộng 2.
- ✓ Khối sau bắt đầu ngay sau địa chỉ quảng bá của khối trước.
- ✓ Sai lầm phổ biến: 62 máy vừa đủ /26, nhưng 63 máy phải dùng /25.

37 Tổng hợp tuyến

Gộp nhiều mạng thành một tuyến ngắn hơn, giúp bảng định tuyến nhỏ lại.

```
import ipaddress as ip

nets = [ip.ip_network(f"172.16.{i}.0/24") for i in range(4)]
print(list(ip.collapse_addresses(nets)))

for i in range(4):
    print(f"172.16.{i:08b}.0")
```

KẾT QUẢ KHI CHẠY

```
[IPv4Network('172.16.0.0/22')]
172.16.00000000.0
172.16.00000001.0
172.16.00000010.0
172.16.00000011.0
```

1. **Đổi octet khác nhau sang nhị phân**
   Viết các giá trị 0, 1, 2, 3 như trên.

2. **Đếm số bit chung bên trái**
   Octet thứ 3 chung 6 bit. Tổng là 8 + 8 + 6 = 22 bit.

3. **Giữ bit chung, còn lại để 0**
   Kết quả 172.16.0.0/22.

Chỉ gộp đúng nếu tuyến tổng không bao trùm mạng không thuộc về bạn.

38 Phân mảnh IP

Gói lớn hơn MTU của chặng sau sẽ bị chia nhỏ. Offset tính theo đơn vị 8 byte.

```
def fragments(total, mtu, header=20):
    data = total - header
    per = (mtu - header) // 8 * 8    # chia hết cho 8
    out, off = [], 0
    while data > 0:
        size = min(per, data)
        data -= size
        out.append((size + header, off // 8, int(data > 0)))
        off += size
    return out

print("Tổng, Offset, MF")
for f in fragments(4000, 1500):
    print(f)
```

KẾT QUẢ KHI CHẠY

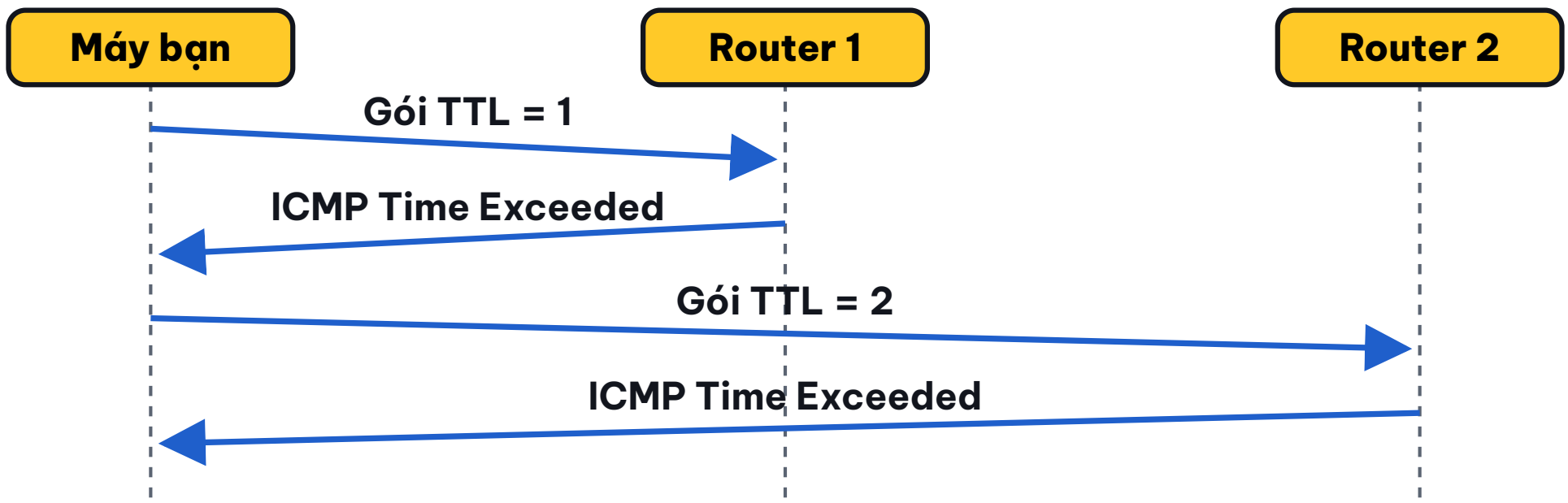
```
Tổng, Offset, MF
(1500, 0, 1)
(1500, 185, 1)
(1040, 370, 0)
```

- Bit MF bằng 1 nghĩa là còn mảnh phía sau. Mảnh cuối có MF bằng 0.
- Các mảnh dùng chung Identification. Máy đích ghép lại, router không ghép.
- Bit DF bằng 1 thì không được phân mảnh, router sẽ hủy gói và báo ICMP.
- IPv6 không cho router phân mảnh, máy gửi phải tự dò MTU.

39 ICMP, ping, traceroute

ICMP mang thông báo lỗi và thông tin điều khiển của tầng mạng.

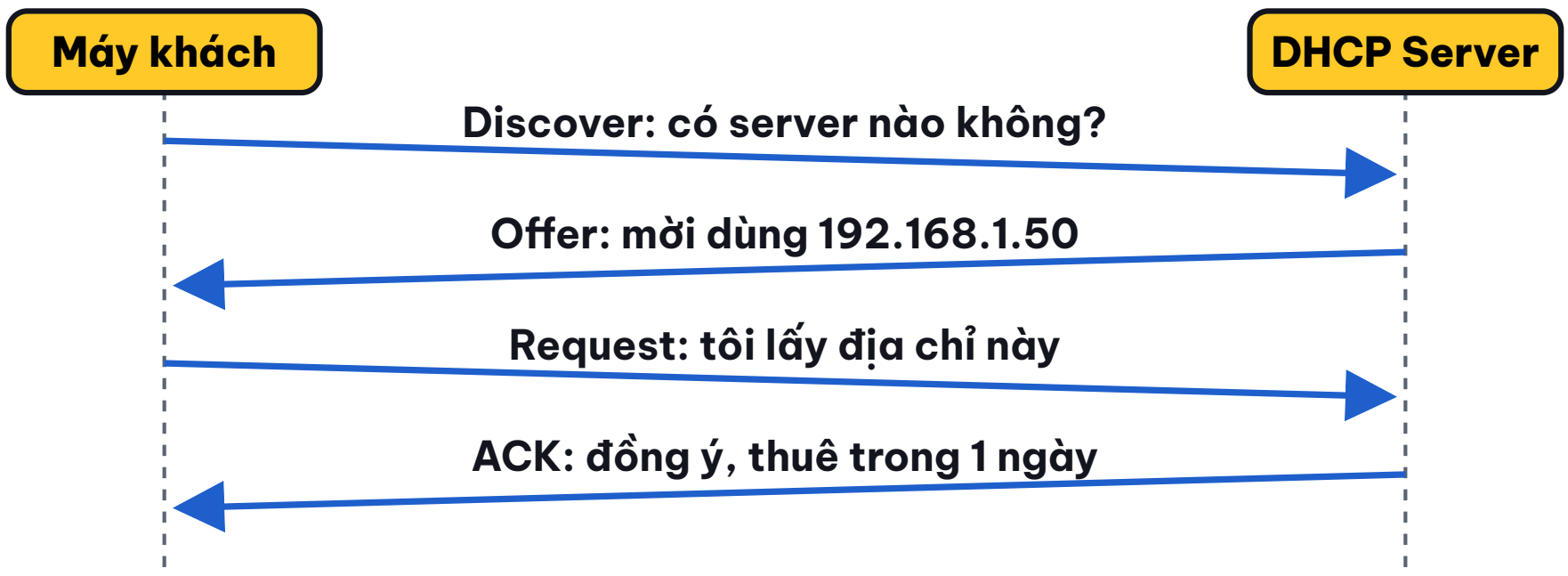
Type	Ý nghĩa	Gặp ở
8 và 0	Echo Request, Echo Reply	ping
3	Không tới được đích	Không có đường, cổng đóng
11	Hết TTL	traceroute



- Traceroute tăng TTL dần để lần lượt lộ ra từng router trên đường.
- Windows dùng `tracert`, Linux dùng `traceroute`.
- Ping không thông chưa chắc máy kia tắt, vì firewall có thể chặn ICMP.

40 DHCP

Máy vào mạng tự nhận IP, mặt nạ, gateway và DNS qua 4 bước, nhớ tắt là DORA.



- Chạy trên UDP: server dùng cổng 67, máy khách dùng cổng 68.
- Discover và Request thường gửi quảng bá, vì máy chưa có địa chỉ và có thể có nhiều server.
- Thuê hết hạn thì phải gia hạn. Máy thường xin gia hạn khi hết nửa thời gian.
- Server ở mạng khác thì router cần bật DHCP relay.
- Không nhận được DHCP, Windows tự gán dải 169.254.x.x.

```
Router(config)# ip dhcp excluded-address 192.168.1.1 192.168.1.10
Router(config)# ip dhcp pool LAN
Router(dhcp-config)# network 192.168.1.0 255.255.255.0
Router(dhcp-config)# default-router 192.168.1.1
Router(dhcp-config)# dns-server 8.8.8.8
```

41 NAT và PAT

Nhiều máy dùng địa chỉ riêng cùng đi ra Internet qua một địa chỉ công cộng. Router phân biệt bằng số cổng.

```
class NAT:
    def __init__(self, public):
        self.public, self.port, self.table = public, 40000, {}

    def out(self, ip, port):                    # đi ra Internet
        if (ip, port) not in self.table:
            self.table[(ip, port)] = self.port
            self.port += 1
        return self.public, self.table[(ip, port)]

    def back(self, port):                       # phản hồi đi vào
        return next(k for k, v in self.table.items() if v == port)

nat = NAT("203.0.113.5")
print(nat.out("192.168.1.10", 5000))
print(nat.out("192.168.1.11", 5000))
print(nat.back(40001))
```

KẾT QUẢ KHI CHẠY

```
('203.0.113.5', 40000)
('203.0.113.5', 40001)
('192.168.1.11', 5000)
```

Loại	Ánh xạ	Dùng khi
Static NAT	1 riêng : 1 công cộng	Đưa server ra Internet
Dynamic NAT	Lấy từ một nhóm địa chỉ	Có nhiều IP công cộng
PAT (NAT overload)	Nhiều riêng : 1 công cộng	Gia đình, doanh nghiệp

Từ ngoài vào không tự mở được kết nối, trừ khi cấu hình chuyển tiếp cổng.

42 IPv6: địa chỉ

IPv6 dài 128 bit, viết thành 8 nhóm hệ 16. Có 2 luật rút gọn hay ra đề.

```
import ipaddress as ip

a = ip.ip_address("2001:0db8:0000:0000:0000:ff00:0042:8329")
print(a.compressed)
print(ip.ip_address("2001:db8::1").exploded)
print(ip.ip_address("fe80::1").is_link_local)
print(2 ** 128)
```

KẾT QUẢ KHI CHẠY

```
2001:db8::ff00:42:8329
2001:0db8:0000:0000:0000:0000:0000:0001
True
340282366920938463463374607431768211456
```

1 Bỏ số 0 đứng đầu mỗi nhóm

`0db8` thành `db8`, `0042` thành `42`.

2 Một dãy nhóm 0 thay bằng ::

Chỉ được dùng một lần trong mỗi địa chỉ, nên chọn dãy dài nhất.

Dải	Loại	Tương tự IPv4
2000::/3	Global unicast	Địa chỉ công cộng
fe80::/10	Link-local	Chỉ dùng trong một chặng
fc00::/7	Unique local	Dải riêng
ff00::/8	Multicast	Thay thế quảng bá
::1/128	Loopback	127.0.0.1

43 IPv6: hoạt động

Header đơn giản hơn, không còn quảng bá, máy có thể tự cấu hình địa chỉ.

```python
def eui64(mac):                                  # tạo Interface ID từ MAC
    b = bytearray.fromhex(mac.replace(":", ""))
    b[0] ^= 2                                    # đảo bit thứ 7
    b = b[:3] + b"\xff\xfe" + b[3:]              # chèn fffe vào giữa
    return ":".join(b[i:i + 2].hex() for i in range(0, 8, 2))

print(eui64("00:1a:2b:3c:4d:5e"))
```

KẾT QUẢ KHI CHẠY

```
021a:2bff:fe3c:4d5e
```

Khái niệm	Ý nghĩa
SLAAC	Máy tự tạo địa chỉ từ tiền tố router quảng bá
NDP	Thay ARP, dùng ICMPv6 và multicast
Dual stack	Máy chạy song song IPv4 và IPv6
Tunneling	Bọc gói IPv6 trong IPv4 để đi qua mạng cũ
NAT64	Dịch giữa IPv6 và IPv4

BỎ ĐI
Checksum trong header, phân mảnh tại router, quảng bá.

THÊM VÀO
Flow Label, Extension Header, không gian địa chỉ rất lớn.

Hệ điều hành hiện đại thường dùng địa chỉ ngẫu nhiên thay cho EUI-64 để tránh lộ MAC.

44 Router làm gì?

Chuyển tiếp là đưa gói từ cổng vào sang cổng ra. Định tuyến là tính ra bảng để biết phải đi cổng nào.

```
import ipaddress as ip

table = [("0.0.0.0/0", "ISP"), ("10.0.0.0/8", "R2"),
         ("10.1.0.0/16", "R3"), ("10.1.2.0/24", "R4")]

def lookup(dst):
    hits = [(ip.ip_network(n), hop) for n, hop in table
            if ip.ip_address(dst) in ip.ip_network(n)]
    return max(hits, key=lambda x: x[0].prefixlen)[1]

for d in ("10.1.2.3", "10.1.9.9", "10.9.9.9", "8.8.8.8"):
    print(d, "->", lookup(d))
```

KẾT QUẢ KHI CHẠY

```
10.1.2.3 -> R4
10.1.9.9 -> R3
10.9.9.9 -> R2
8.8.8.8 -> ISP
```

Quy tắc so khớp dài nhất: nhiều tuyến cùng khớp thì chọn tuyến có prefix dài nhất. Tuyến 0.0.0.0/0 là tuyến mặc định, chỉ dùng khi không còn tuyến nào khác.

- ✓ Router giảm TTL, tính lại checksum, đóng khung mới cho chặng sau.
- ✓ Không có tuyến khớp thì gói bị hủy và có thể báo ICMP.
- ✓ Mặt phẳng điều khiển tính tuyến, mặt phẳng dữ liệu chuyển gói.

45 Bảng định tuyến

Router có thể học tuyến từ nhiều nguồn. Nguồn nào đáng tin hơn có khoảng cách quản trị nhỏ hơn.

```
R1(config)# ip route 192.168.2.0 255.255.255.0 10.0.0.2
R1(config)# ip route 0.0.0.0 0.0.0.0 203.0.113.1
R1# show ip route
```

Mã	Nguồn tuyến
C	Nối trực tiếp
L	Địa chỉ của chính router
S	Tĩnh
S*	Tĩnh mặc định
O	OSPF
R	RIP
B	BGP

Nguồn	AD
Nối trực tiếp	0
Tĩnh	1
eBGP	20
EIGRP	90
OSPF	110
RIP	120
iBGP	200

Thứ tự chọn: prefix dài nhất trước, rồi AD nhỏ nhất, rồi metric nhỏ nhất.

AD là giá trị mặc định của Cisco, có ý nghĩa cục bộ trên router. Metric chỉ so sánh được trong cùng một giao thức.

46 Hai họ định tuyến

	Vector khoảng cách	Trạng thái liên kết
Ví dụ	RIP	OSPF, IS-IS
Biết gì?	Khoảng cách qua hàng xóm	Bản đồ toàn mạng
Gửi gì?	Bảng định tuyến	Trạng thái liên kết
Gửi cho ai?	Hàng xóm	Toàn vùng
Thuật toán	Bellman-Ford	Dijkstra
Hội tụ	Chậm hơn, có thể lặp	Nhanh hơn
Tài nguyên	Ít	Nhiều hơn

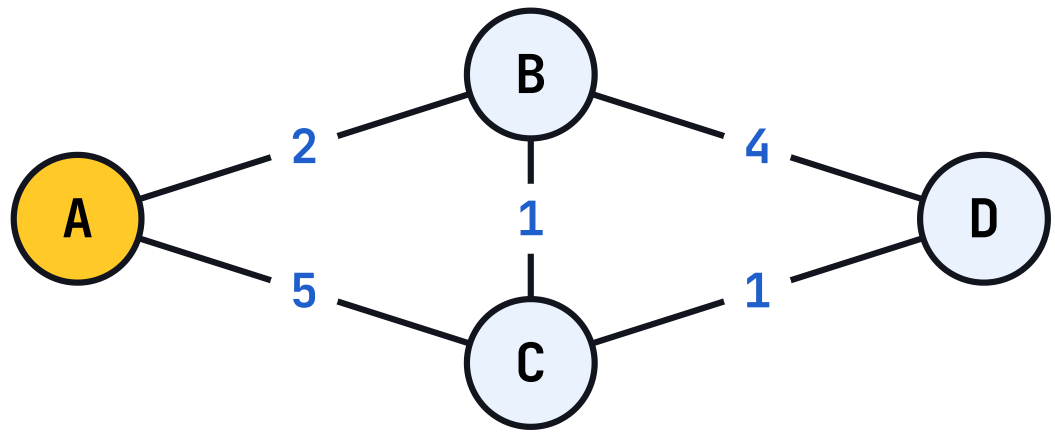
Ví dụ dễ nhớ: vector khoảng cách là hỏi đường người đi trước, trạng thái liên kết là ai cũng cầm bản đồ.

IGP
Định tuyến trong một AS: RIP, OSPF, IS-IS, EIGRP.

EGP
Định tuyến giữa các AS: BGP.

47 Dijkstra trong OSPF

Mỗi router có cùng bản đồ, rồi tự tính cây đường đi ngắn nhất từ chính nó.




```python
import heapq

def dijkstra(g, src):
    dist, prev = {src: 0}, {}
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in g[u].items():
            if d + w < dist.get(v, float("inf")):
                dist[v], prev[v] = d + w, u
                heapq.heappush(pq, (d + w, v))
    return dist, prev

g = {"A": {"B": 2, "C": 5}, "B": {"A": 2, "C": 1, "D": 4},
     "C": {"A": 5, "B": 1, "D": 1}, "D": {"B": 4, "C": 1}}
print(*dijkstra(g, "A"), sep="\n")
```

KẾT QUẢ KHI CHẠY

```
{'A': 0, 'B': 2, 'C': 3, 'D': 4}
{'B': 'A', 'C': 'B', 'D': 'C'}
```

Đường A tới D: A, B, C, D với chi phí 4. `prev` cho biết nút liền trước, nên lần ngược ra được chặng kế tiếp.

48 Bellman-Ford trong RIP

Lấy bảng của hàng xóm, cộng chi phí tới hàng xóm, giữ giá trị nhỏ nhất.

```
INF = float("inf")
g = {"A": {"B": 2, "C": 5}, "B": {"A": 2, "C": 1, "D": 4},
     "C": {"A": 5, "B": 1, "D": 1}, "D": {"B": 4, "C": 1}}

dv = {u: {v: 0 if u == v else g[u].get(v, INF) for v in g} for u in g}
for step in (1, 2):
    dv = {u: {v: min([dv[u][v]] + [g[u][n] + dv[n][v] for n in g[u]])
              for v in g} for u in g}
    print("Vòng", step, dv["A"])

a, b = 2, 1                          # chuỗi A - B - C, đường B - C vừa đứt
for step in range(1, 5):
    b = a + 1                        # B tưởng A còn đường tới C
    a = b + 1
    print("Đếm tới vô cùng:", b, a)
```

KẾT QUẢ KHI CHẠY

```
Vòng 1 {'A': 0, 'B': 2, 'C': 3, 'D': 6}
Vòng 2 {'A': 0, 'B': 2, 'C': 3, 'D': 4}
Đếm tới vô cùng: 3 4
Đếm tới vô cùng: 5 6
Đếm tới vô cùng: 7 8
Đếm tới vô cùng: 9 10
```

Công thức: $d_x(y) = \min\{ c(x, v) + d_v(y) \}$ trên mọi hàng xóm v.

Cách chống lặp	Ý tưởng
Split horizon	Không quảng bá tuyến về phía đã học được nó
Poison reverse	Báo lại với giá trị vô cực
Giới hạn hop	RIP coi 16 là không tới được

49 RIP và OSPF

```
ref = 100e6                          # băng thông tham chiếu mặc định
links = {"10 Mbps": 10e6, "100 Mbps": 100e6, "1 Gbps": 1e9}
for name, bw in links.items():
    print(name, "cost:", max(1, int(ref / bw)))
```

KẾT QUẢ KHI CHẠY

```
10 Mbps cost: 10
100 Mbps cost: 1
1 Gbps cost: 1
```

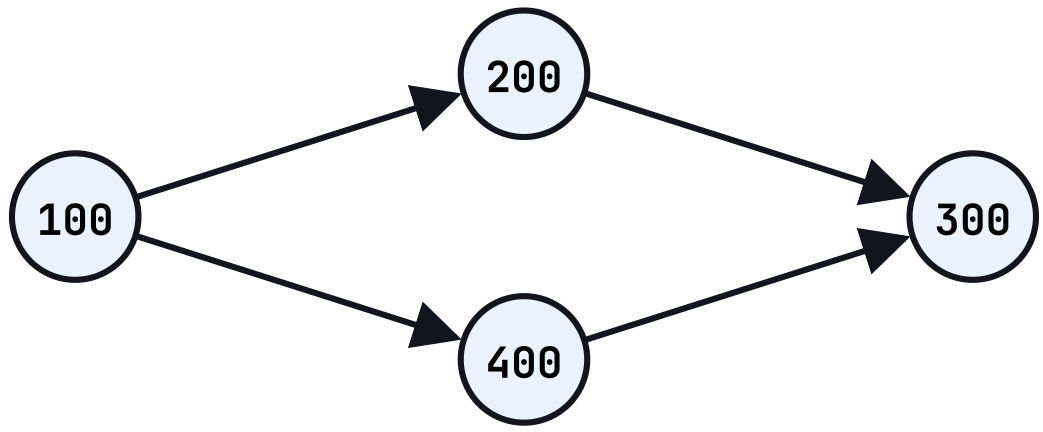
	RIPv2	OSPFv2
Metric	Số hop, tối đa 15	Cost theo băng thông
Cập nhật	Định kỳ 30 giây	Khi có thay đổi
Chạy trên	UDP 520	IP giao thức số 89
Quy mô	Mạng nhỏ	Mạng lớn, chia vùng

```
R1(config)# router ospf 1
R1(config-router)# router-id 1.1.1.1
R1(config-router)# network 192.168.1.0 0.0.0.255 area 0
R1(config-router)# network 10.0.0.0 0.0.0.3 area 0
R1# show ip ospf neighbor
```

- ✓ Wildcard là phần bù của mặt nạ: 255.255.255.0 thành 0.0.0.255.
- ✓ Area 0 là vùng xương sống, các vùng khác phải nối vào nó.
- ✓ Trên mạng đa truy nhập, OSPF bầu DR và BDR để giảm trao đổi.
- ✓ Hàng xóm chỉ hình thành khi khớp vùng, mạng con và các bộ đếm hello, dead.
- ✓ Link 1 Gbps và 100 Mbps cùng cost 1, nên cần tăng băng thông tham chiếu.

50 BGP: định tuyến giữa các AS

BGP là giao thức nối các mạng lớn của Internet. Nó ưu tiên chính sách, không chỉ đường ngắn nhất.




```
routes = {"qua R1": ["AS100", "AS200", "AS300"],
          "qua R2": ["AS400", "AS300"]}

print(min(routes, key=lambda r: len(routes[r])))

path = ["AS100", "AS200", "AS300"]
print("AS100" in path)      # thấy AS của mình: có vòng, bỏ tuyến
```

KẾT QUẢ KHI CHẠY

```
qua R2
True
```

Khái niệm	Ý nghĩa
AS	Mạng dưới một chính sách quản trị
Path vector	Quảng bá cả danh sách AS đã đi qua
eBGP, iBGP	Giữa các AS, và trong một AS
Phiên BGP	Chạy trên TCP cổng 179

BGP thực tế xét nhiều thuộc tính, LOCAL_PREF được xét trước AS_PATH.

51 Định tuyến giữa các VLAN

Máy ở VLAN khác nhau phải đi qua thiết bị tầng 3. Hai cách phổ biến là router on a stick và switch tầng 3.

```
Router(config)# interface gi0/0
Router(config-if)# no shutdown
Router(config)# interface gi0/0.10
Router(config-subif)# encapsulation dot1Q 10
Router(config-subif)# ip address 192.168.10.1 255.255.255.0
Router(config)# interface gi0/0.20
Router(config-subif)# encapsulation dot1Q 20
Router(config-subif)# ip address 192.168.20.1 255.255.255.0
```

```
Switch(config)# ip routing
Switch(config)# interface vlan 10
Switch(config-if)# ip address 192.168.10.1 255.255.255.0
Switch(config-if)# no shutdown
```

Cách	Ưu điểm	Nhược điểm
Router on a stick	Rẻ, đơn giản	Một đường dễ nghẽn
Switch tầng 3	Nhanh, hợp mạng lớn	Thiết bị đắt hơn

Cổng switch nối router phải đặt trunk. Gateway của mỗi VLAN là địa chỉ của subinterface tương ứng.

DỰ PHÒNG GATEWAY (FHRP)
HSRP, VRRP, GLBP cho nhiều router dùng chung một gateway ảo. Router chính hỏng thì router dự phòng thay thế, máy tính không phải đổi cấu hình.

52 Tầng giao vận làm gì?

IP đưa gói tới đúng máy. Tầng giao vận đưa dữ liệu tới đúng ứng dụng, nhờ số cổng.

Dải cổng	Ý nghĩa
0 đến 1023	Cổng phổ biến của dịch vụ hệ thống
1024 đến 49151	Cổng đăng ký
49152 đến 65535	Cổng động cho phía máy khách

PHÂN KÊNH UDP

Dựa vào IP đích và cổng đích.

PHÂN KÊNH TCP

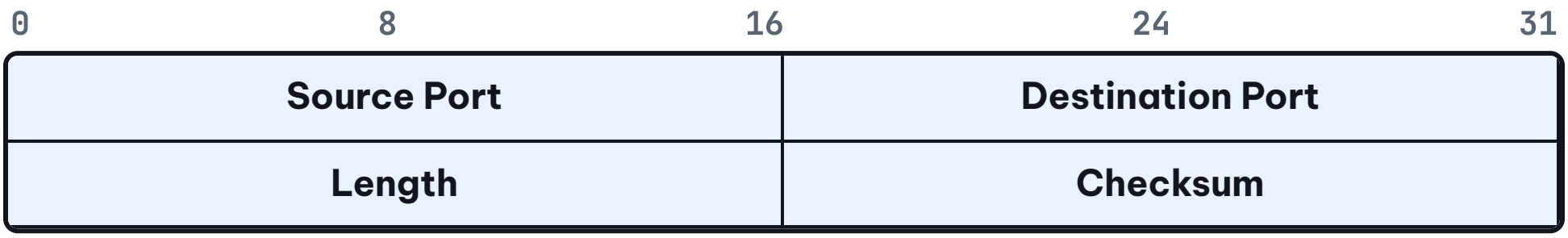
Dựa vào 4 giá trị: IP nguồn, cổng nguồn, IP đích, cổng đích.

Khái niệm	Ý nghĩa
Socket	Điểm cuối ứng dụng dùng để gửi, nhận
Dồn kênh	Gom dữ liệu nhiều ứng dụng xuống mạng
Phân kênh	Trả dữ liệu về đúng ứng dụng

Nhờ 4 giá trị này mà một web server cổng 443 phục vụ cùng lúc rất nhiều kết nối.

53 UDP

Nhẹ và nhanh: không kết nối, không gửi lại, không đảm bảo thứ tự. Tầng ứng dụng tự lo nếu cần.




```python
import socket, threading

srv = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
srv.bind(("127.0.0.1", 0))
port = srv.getsockname()[1]

def serve():
    data, addr = srv.recvfrom(1024)
    srv.sendto(data.upper(), addr)

threading.Thread(target=serve).start()

cli = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
cli.sendto(b"xin chao", ("127.0.0.1", port))
print(cli.recvfrom(1024)[0])
```

KẾT QUẢ KHI CHẠY

```
b'XIN CHAO'
```

- Header chỉ 8 byte, so với tối thiểu 20 byte của TCP.
- Dùng cho DNS, DHCP, streaming, game, VoIP, và là nền của QUIC.
- Không có bắt tay, nên có thể gửi ngay, nhưng không biết bên kia đã nhận chưa.

54 Truyền tin cậy: Stop-and-Wait

Gửi một gói, chờ ACK rồi mới gửi tiếp. Đơn giản, nhưng rất lãng phí đường truyền khi RTT lớn.

```python
def util(L, R, rtt, window=1):
    t = L / R                              # thời gian đẩy một gói
    return min(1, window * t / (rtt + t))

L, R, rtt = 8000, 1e9, 0.03                # 1 KB, 1 Gbps, RTT 30 ms

print(f"Stop-and-Wait: {util(L, R, rtt):.5f}")
print(f"Cửa sổ 3 gói:  {util(L, R, rtt, 3):.5f}")
print(f"Cửa sổ 3751:   {util(L, R, rtt, 3751):.5f}")
```

KẾT QUẢ KHI CHẠY

```
Stop-and-Wait: 0.00027
Cửa sổ 3 gói:  0.00080
Cửa sổ 3751:   1.00000
```

Cơ chế	Giải quyết
Số thứ tự	Phát hiện gói trùng, gói mất
ACK	Xác nhận đã nhận
Bộ đếm thời gian	Hết giờ thì gửi lại
Checksum	Phát hiện lỗi bit
Cửa sổ trượt	Gửi nhiều gói khi chưa có ACK

Công thức bài thi: U = (L / R) / (RTT + L / R). Cửa sổ N gói thì nhân thêm N, tối đa bằng 1.

55 Go-Back-N và Selective Repeat

Cửa sổ trượt cho phép gửi nhiều gói. Khác nhau ở cách xử lý khi một gói bị mất.

```
def go_back_n(n, w, lost):          # mô hình đơn giản theo từng lượt
    lost, sent, base = set(lost), 0, 0
    while base < n:
        ok, nxt = True, base
        for i in range(base, min(base + w, n)):
            sent += 1
            if i in lost:
                lost.discard(i)
                ok = False          # các gói sau bị bên nhận bỏ
            elif ok:
                nxt = i + 1
        base = nxt
    return sent

def selective_repeat(n, w, lost):
    return n + len(set(lost))    # chỉ gửi lại gói mất

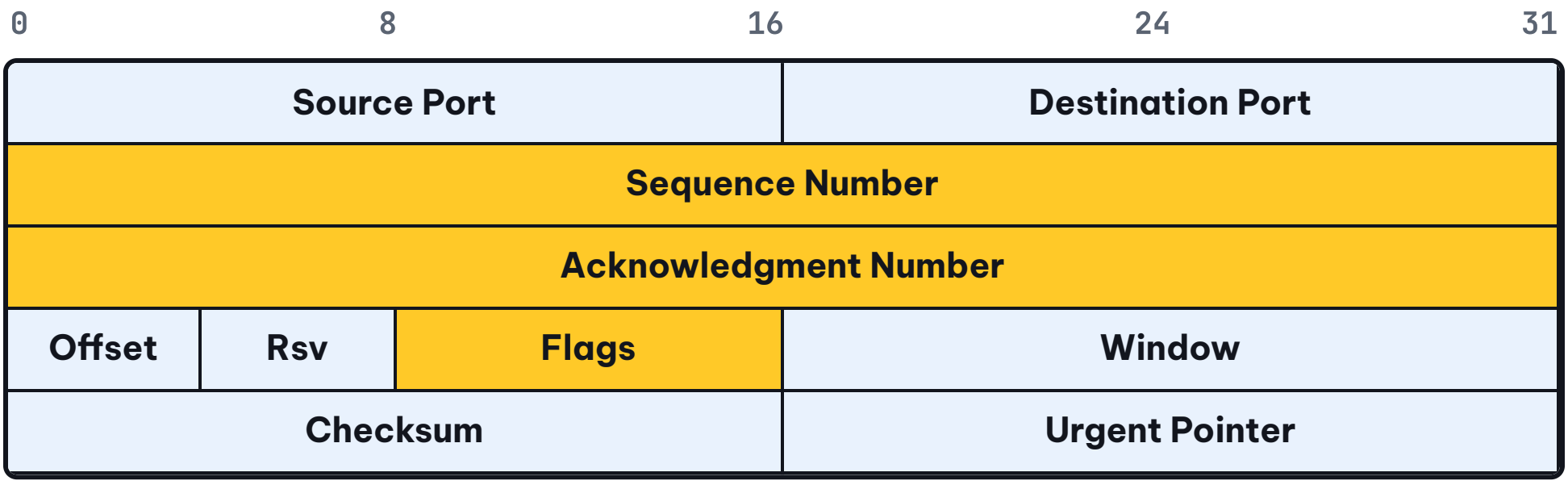
print(go_back_n(10, 4, [2]), selective_repeat(10, 4, [2]))
```

KẾT QUẢ KHI CHẠY

```
12 11
```

	Go-Back-N	Selective Repeat
ACK	Tích lũy	Từng gói
Bên nhận	Bỏ gói sai thứ tự	Lưu đệm gói sai thứ tự
Gửi lại	Từ gói mất trở đi	Chỉ gói mất
Cửa sổ tối đa, k bit	2^k - 1	2^(k - 1)

56 Header TCP




```python
import struct

raw = bytes.fromhex("c35001bb000000640000000005002fffe00000000")
sp, dp, seq, ack, of, win, cs, up = struct.unpack("!HHIIHHHH", raw)

names = "FIN SYN RST PSH ACK URG ECE CWR".split()
print("Cổng:", sp, "->", dp, "| seq:", seq, "| cửa sổ:", win)
print("Header:", (of >> 12) * 4, "byte")
print("Cờ:", [n for i, n in enumerate(names) if of >> i & 1])
```

KẾT QUẢ KHI CHẠY

```
Cổng: 50000 -> 443 | seq: 100 | cửa sổ: 65534
Header: 20 byte
Cờ: ['SYN']
```

Cờ	Ý nghĩa
SYN	Xin mở kết nối, đồng bộ số thứ tự
ACK	Trường Acknowledgment có hiệu lực
FIN	Không còn dữ liệu để gửi
RST	Hủy kết nối ngay
PSH	Đẩy dữ liệu lên ứng dụng sớm

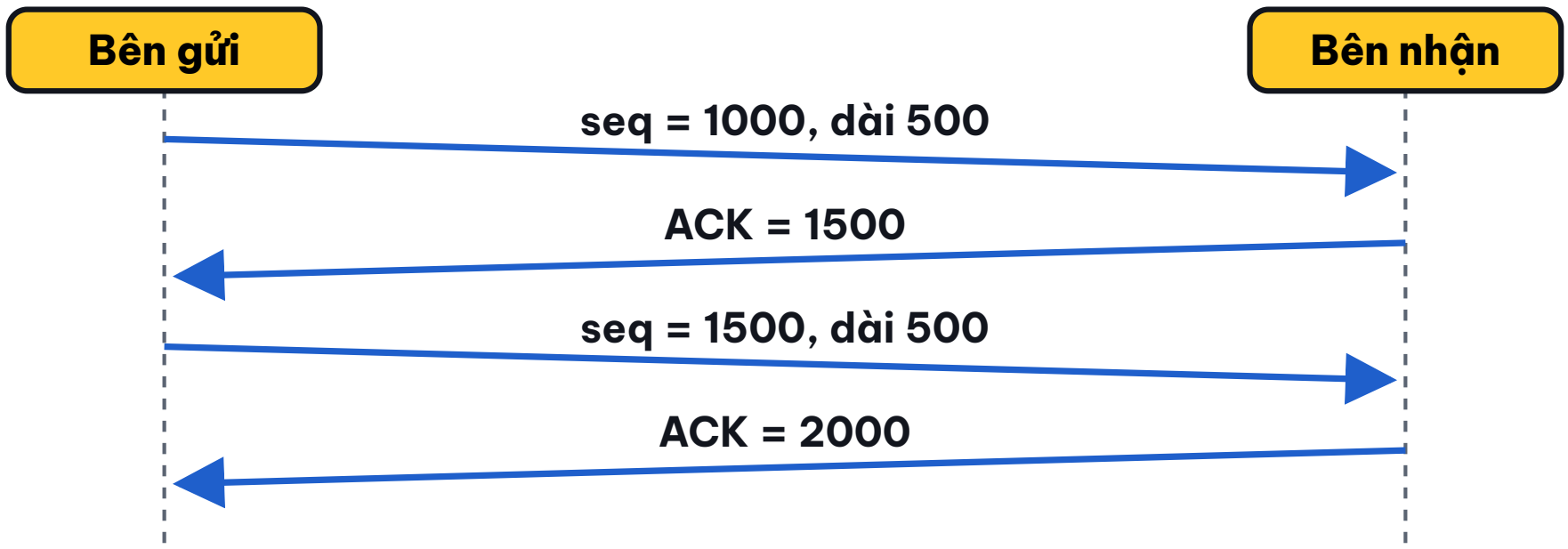
57 Số thứ tự và ACK

TCP đánh số theo từng byte, không theo từng gói. Số ACK là byte tiếp theo mà bên nhận đang chờ.

```
seq = 1000
for size in (500, 500, 200):
    print("Gửi seq", seq, "dài", size, "-> ACK", seq + size)
    seq += size
```

KẾT QUẢ KHI CHẠY

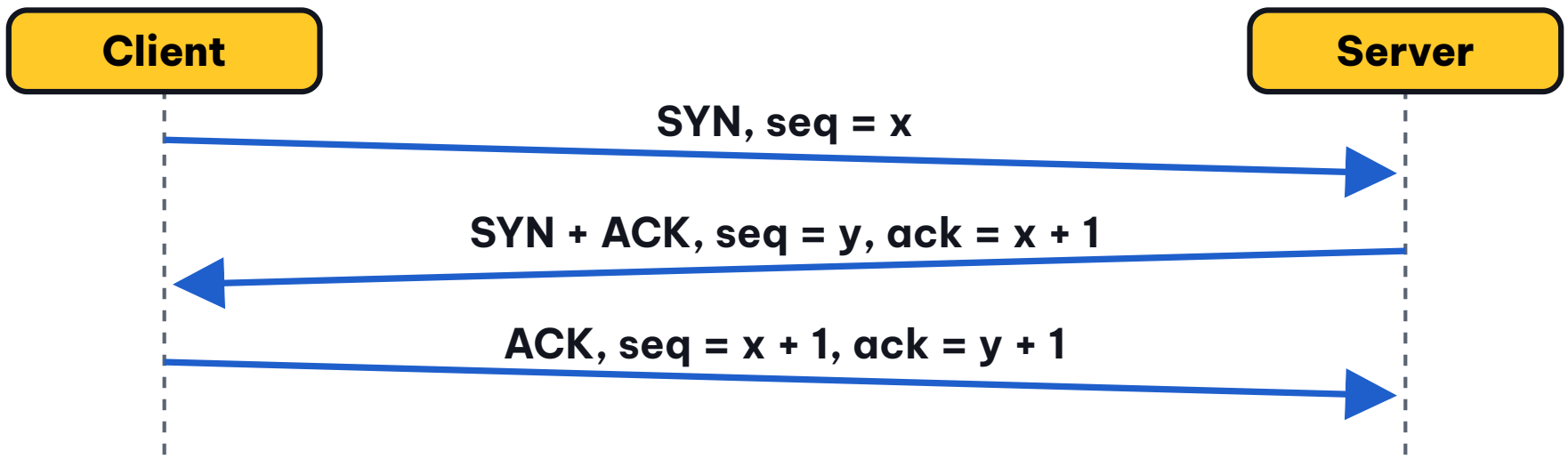
```
Gửi seq 1000 dài 500 -> ACK 1500
Gửi seq 1500 dài 500 -> ACK 2000
Gửi seq 2000 dài 200 -> ACK 2200
```



- ACK tích lũy: ACK 2000 nghĩa là đã nhận đủ mọi byte trước 2000.
- Nhận gói sai thứ tự thì lặp lại ACK cũ, tạo ra ACK trùng.
- 3 ACK trùng kích hoạt gửi lại nhanh, không cần chờ hết giờ.
- MSS là phần dữ liệu tối đa trong một đoạn, thường 1.460 byte khi MTU là 1.500.

58 Bắt tay 3 bước

Hai bên thống nhất số thứ tự ban đầu trước khi gửi dữ liệu.



- SYN chiếm một số thứ tự dù không mang dữ liệu, nên ACK là x + 1.
- Số thứ tự ban đầu được chọn ngẫu nhiên, để khó đoán và tránh nhầm với kết nối cũ.
- Cần 3 bước để cả hai bên đều chắc chiều gửi và chiều nhận của mình hoạt động.

BÀI THI HAY CHO

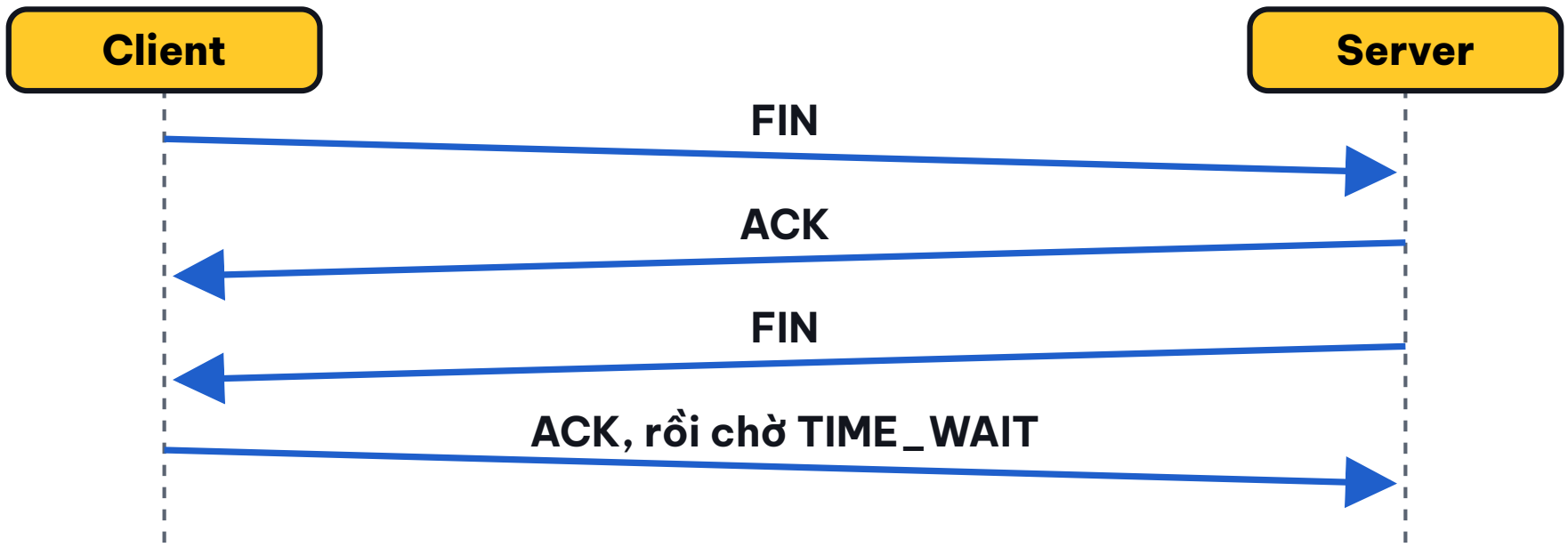
Cho seq = 100, hỏi ACK của bước 2. Đáp án là 101.

SYN FLOOD

Kẻ tấn công gửi nhiều SYN mà không hoàn tất, làm đầy hàng đợi. SYN cookie giúp giảm nhẹ.

59 Đóng kết nối TCP

Mỗi chiều đóng riêng, nên thường có 4 bước.



Trạng thái	Ý nghĩa
LISTEN	Server đang chờ kết nối
SYN-SENT	Client đã gửi SYN
ESTABLISHED	Đã kết nối, đang trao đổi dữ liệu
CLOSE-WAIT	Đã nhận FIN, ứng dụng chưa đóng
TIME-WAIT	Chờ 2 MSL để gói cũ hết hẳn

TIME_WAIT giúp gửi lại ACK cuối nếu bị mất, và tránh gói cũ lọt vào kết nối mới.

Xem trạng thái kết nối bằng `netstat -an` trên Windows, hoặc `ss -tan` trên Linux.

60 Kiểm soát luồng

Bên nhận báo còn bao nhiêu chỗ trống qua trường Window, để bên gửi không làm tràn bộ đệm.

```
def max_throughput(window_bytes, rtt):
    return window_bytes * 8 / rtt / 1e6        # Mbps

print(round(max_throughput(65535, 0.1), 2), "Mbps, RTT 100 ms")
print(round(max_throughput(65535, 0.01), 2), "Mbps, RTT 10 ms")

need = 1e9 * 0.1 / 8                          # 1 Gbps, RTT 100 ms
print("Cửa sổ cần:", int(need), "byte")
```

KẾT QUẢ KHI CHẠY

```
5.24 Mbps, RTT 100 ms
52.43 Mbps, RTT 10 ms
Cửa sổ cần: 12500000 byte
```

- Công thức: thông lượng tối đa ≈ cửa sổ / RTT.
- Trường Window 16 bit chỉ biểu diễn tới 65.535 byte, nên có tùy chọn Window Scale.
- Window bằng 0 thì bên gửi dừng, và thỉnh thoảng gửi gói thăm dò.
- Bên gửi bị giới hạn bởi min(rwnd, cwnd).

KIỂM SOÁT LUỒNG
Bảo vệ bên nhận. Giá trị do bên nhận báo.

KIỂM SOÁT TẮC NGHẼN
Bảo vệ mạng. Giá trị do bên gửi tự ước lượng.

61 Kiểm soát tắc nghẽn

Bên gửi tăng dần tốc độ để dò khả năng của mạng, và giảm khi có dấu hiệu tắc nghẽn.

```
def reno(rounds, ssthresh, loss_at):
    cwnd, log = 1, []
    for t in range(rounds):
        log.append(cwnd)
        if t in loss_at:                    # nhận 3 ACK trùng
            ssthresh = max(cwnd // 2, 2)
            cwnd = ssthresh
        elif cwnd < ssthresh:
            cwnd = min(cwnd * 2, ssthresh)   # slow start
        else:
            cwnd += 1                       # tăng tuyến tính
    return log

print(reno(14, 8, {9}))
```

KẾT QUẢ KHI CHẠY

```
[1, 2, 4, 8, 9, 10, 11, 12, 13, 14, 7, 8, 9, 10]
```

Sự kiện	Phản ứng
Slow start	cwnd gấp đôi mỗi RTT, tới ssthresh
Tránh tắc nghẽn	cwnd tăng 1 MSS mỗi RTT
3 ACK trùng (Reno)	ssthresh = cwnd / 2, cwnd về ssthresh
3 ACK trùng (Tahoe)	ssthresh = cwnd / 2, cwnd về 1
Hết giờ	ssthresh = cwnd / 2, cwnd về 1

Đây là mô hình rút gọn. Giáo trình Kurose cộng thêm 3 MSS khi vào Fast Recovery, còn Linux mặc định dùng CUBIC. Hãy tính theo đúng cách đề yêu cầu.

62 Ước lượng RTT và RTO

Hết giờ quá sớm thì gửi lại thừa, quá muộn thì chậm phát hiện mất gói. TCP đo RTT liên tục để chỉnh bộ đếm.

```python
def rto(samples, a=1 / 8, b=1 / 4):
    srtt, var = samples[0], samples[0] / 2
    for r in samples[1:]:
        var = (1 - b) * var + b * abs(srtt - r)
        srtt = (1 - a) * srtt + a * r
    return round(srtt, 2), round(var, 2), round(srtt + 4 * var, 2)

print(rto([100, 120, 90, 110]))   # SRTT, RTTVAR, RTO (ms)
```

KẾT QUẢ KHI CHẠY

```
(102.07, 28.52, 216.13)
```

Công thức	Ý nghĩa
$SRTT = (1 - \alpha)\cdot SRTT + \alpha\cdot R$	Trung bình trượt, α = 1/8
$RTTVAR = (1 - \beta)\cdot RTTVAR + \beta\cdot\lvert SRTT - R\rvert$	Độ dao động, β = 1/4
$RTO = SRTT + 4\cdot RTTVAR$	Thời gian chờ

- ✓ Theo RFC 6298, RTTVAR được cập nhật trước, dùng SRTT cũ.
- ✓ Không đo RTT trên gói gửi lại, vì không biết ACK ứng với lần gửi nào (thuật toán Karn).
- ✓ Sau mỗi lần hết giờ, RTO tăng gấp đôi.

63 TCP, UDP và QUIC

	TCP	UDP
Kết nối	Có	Không
Tin cậy	Có, gửi lại khi mất	Không
Thứ tự	Đảm bảo	Không
Kiểm soát tắc nghẽn	Có	Không
Header	Từ 20 byte	8 byte
Dữ liệu	Dòng byte	Từng datagram
Ví dụ	Web, email, SSH	DNS, game, gọi video

QUIC
Chạy trên UDP, tự làm phần tin cậy và kiểm soát tắc nghẽn, tích hợp mã hóa TLS 1.3, nhiều luồng độc lập. Đây là nền của HTTP/3.

Không có giao thức tốt nhất. Hãy chọn theo việc ứng dụng sợ gì hơn: mất dữ liệu hay trễ.

64 Lập trình socket TCP

Socket là giao diện lập trình để ứng dụng dùng tầng giao vận. Code dưới đây chạy thật trên máy.

```
import socket, threading

srv = socket.create_server(("127.0.0.1", 0))
port = srv.getsockname()[1]

def serve():
    conn, addr = srv.accept()
    with conn:
        conn.sendall(conn.recv(1024).upper())

t = threading.Thread(target=serve)
t.start()

with socket.create_connection(("127.0.0.1", port)) as c:
    c.sendall(b"xin chao")
    print(c.recv(1024))
t.join()
```

KẾT QUẢ KHI CHẠY

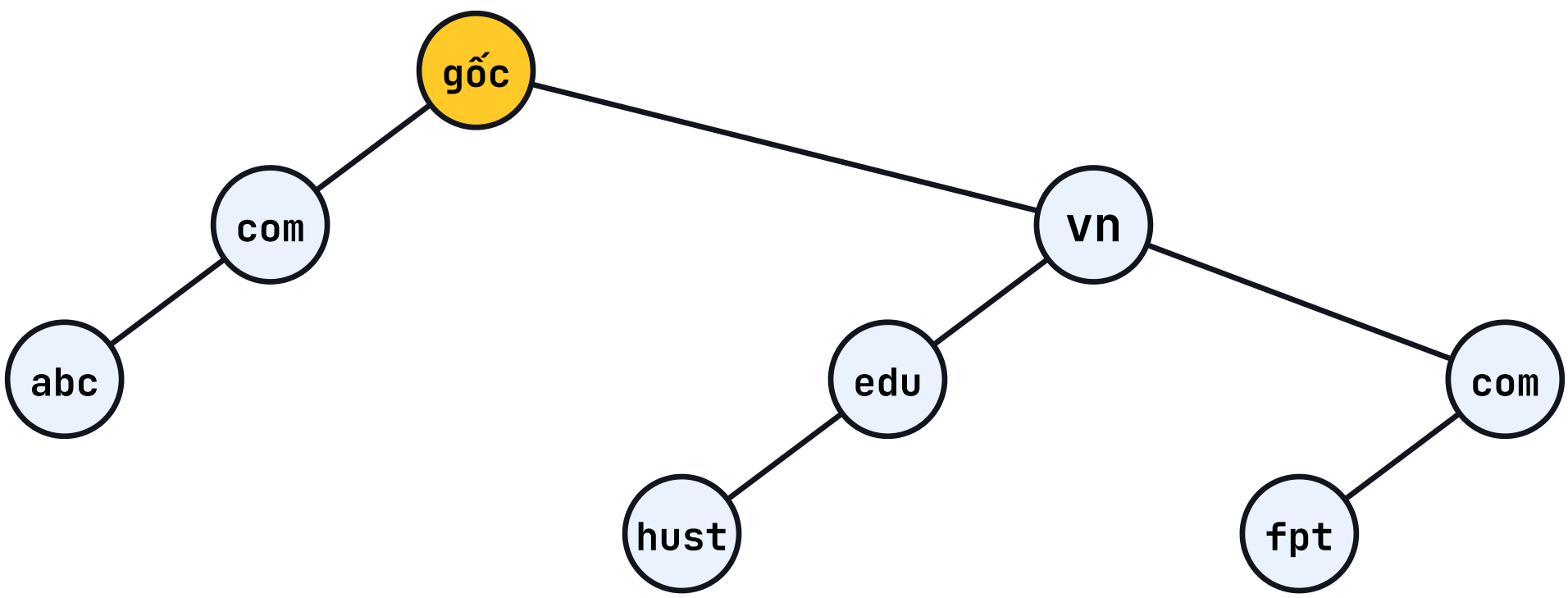
```
b'XIN CHAO'
```

Phía	Các bước
TCP server	socket, bind, listen, accept, recv/send, close
TCP client	socket, connect, send/recv, close
UDP	socket, bind, sendto/recvfrom

- ✓ TCP là dòng byte: một lần `recv` không nhất thiết khớp một lần `send`.
- ✓ Cổng 0 nghĩa là để hệ điều hành chọn một cổng trống.

65 DNS: danh bạ của Internet

DNS đổi tên miền dễ nhớ thành địa chỉ IP. Dữ liệu được phân tán theo cây, không nằm ở một máy chủ duy nhất.

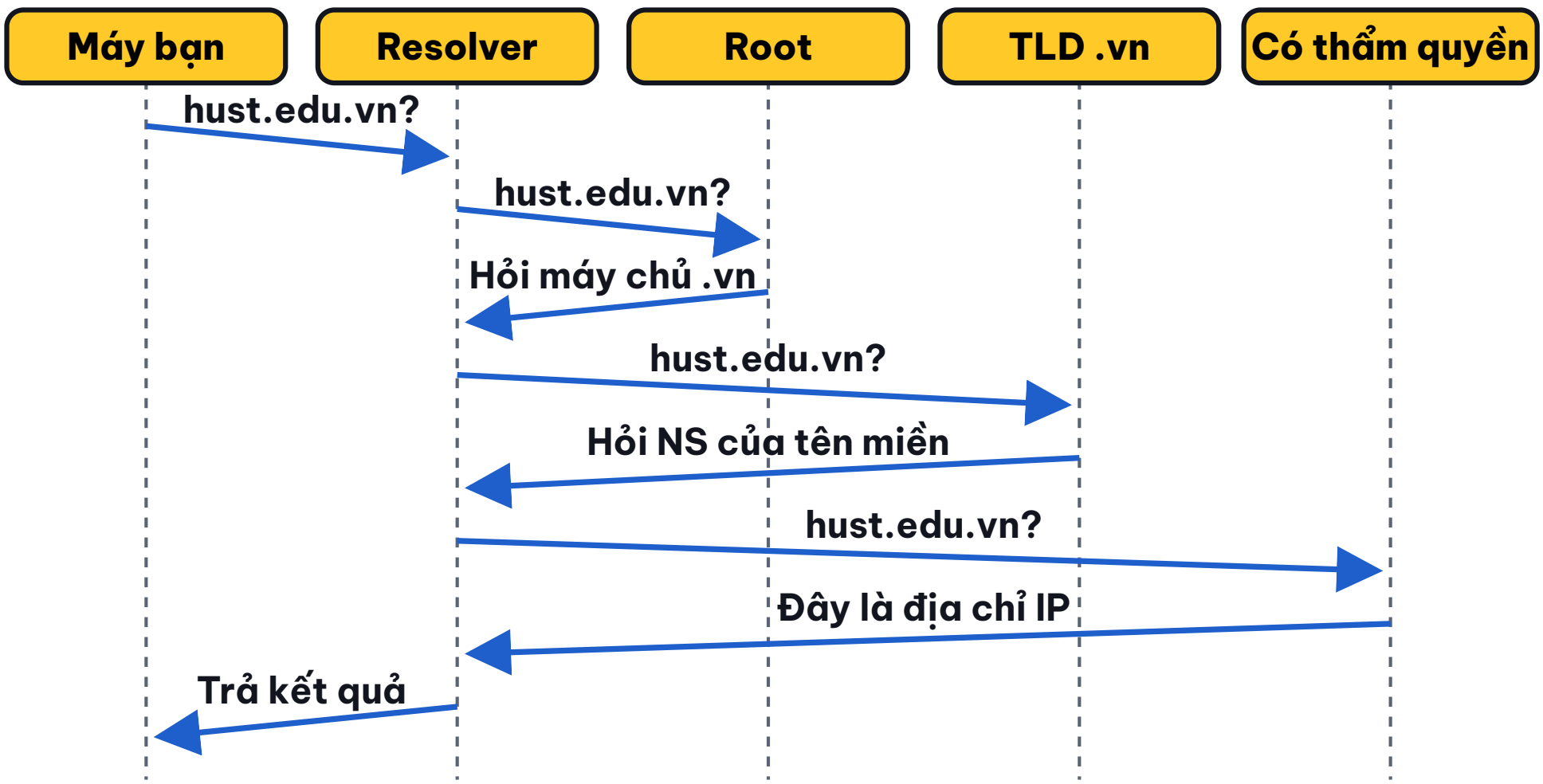


Bản ghi	Ý nghĩa
A	Tên miền trỏ tới IPv4
AAAA	Tên miền trỏ tới IPv6
CNAME	Bí danh của một tên khác
MX	Máy chủ nhận thư
NS	Máy chủ có thẩm quyền của vùng
PTR	Tra ngược từ IP ra tên
TXT	Văn bản, hay dùng để xác thực tên miền, email
SOA	Thông tin quản trị của vùng

DNS chủ yếu dùng UDP cổng 53, và dùng TCP cổng 53 cho phản hồi lớn hoặc truyền vùng.

66 DNS phân giải tên thế nào?

Máy bạn hỏi resolver theo kiểu đệ quy. Resolver đi hỏi từng cấp theo kiểu lặp, rồi lưu vào cache.



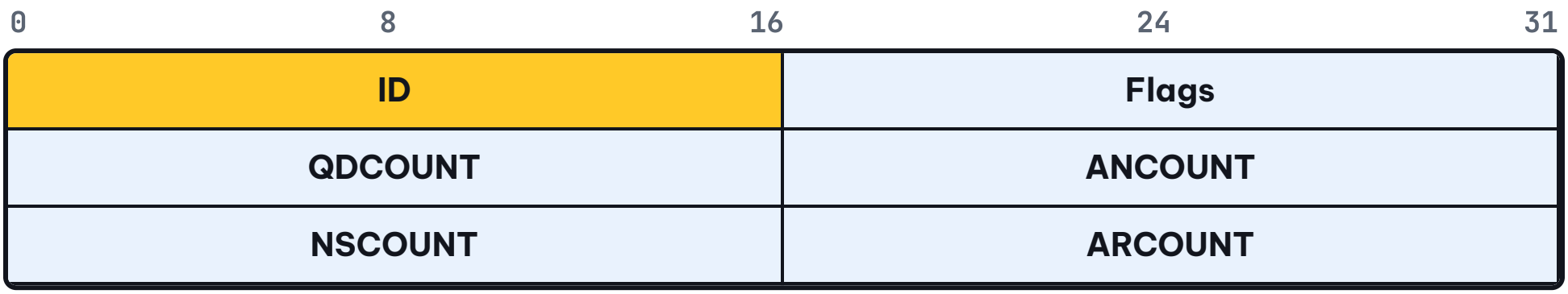
ĐỆ QUY
Hỏi một lần, nhận đáp án cuối cùng.

LẶP
Mỗi máy chủ chỉ đường tới cấp tiếp theo.

TTL quyết định bản ghi được lưu cache bao lâu. Đổi DNS không có hiệu lực ngay ở mọi nơi là vì vậy.

67 Bản tin DNS

Truy vấn và phản hồi dùng chung định dạng, header dài 12 byte.




```python
import struct

def dns_query(name, qid=0x1234):
    header = struct.pack("!HHHHHH", qid, 0x0100, 1, 0, 0, 0)
    parts = [bytes([len(p)]) + p.encode() for p in name.split(".")]
    labels = b"".join(parts)
    return header + labels + b"\0" + struct.pack("!HH", 1, 1)

q = dns_query("vnnic.vn")
print(q[:12].hex(" "))        # header 12 byte
print(q[12:].hex(" "))        # phần câu hỏi
```

KẾT QUẢ KHI CHẠY

```
12 34 01 00 00 01 00 00 00 00 00 00
05 76 6e 6e 69 63 02 76 6e 00 00 01 00 01
```

Giá trị	Ý nghĩa
`12 34`	ID để ghép câu hỏi với câu trả lời
`01 00`	Cờ RD: xin phân giải đệ quy
`00 01`	Có 1 câu hỏi
`05 76 6e 6e 69 63`	Nhãn dài 5 byte: vnnic
`02 76 6e 00`	Nhãn dài 2 byte: vn, rồi byte kết thúc
`00 01 00 01`	Loại A, lớp IN

68 HTTP

Giao thức yêu cầu và phản hồi của web. Máy khách gửi request, máy chủ trả response.

```
import http.server, threading, urllib.request

class Hello(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        body = f"Ban vua goi {self.path}".encode()
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *args):
        pass

srv = http.server.HTTPServer(("127.0.0.1", 0), Hello)
threading.Thread(target=srv.handle_request).start()
url = f"http://127.0.0.1:{srv.server_port}/mang"
with urllib.request.urlopen(url) as r:
    print(r.status, r.headers["Content-Type"], r.read())
```

KẾT QUẢ KHI CHẠY

```
200 text/plain b'Ban vua goi /mang'
```

Phương thức	Dùng để
GET	Lấy dữ liệu
POST	Gửi, tạo mới
PUT	Thay thế
DELETE	Xóa

Mã	Ý nghĩa
2xx	Thành công
3xx	Chuyển hướng
4xx	Lỗi phía khách
5xx	Lỗi phía máy chủ

69 Các phiên bản HTTP

Phiên bản	Điểm chính	Chạy trên
HTTP/1.0	Mỗi yêu cầu một kết nối	TCP
HTTP/1.1	Giữ kết nối, thêm Host	TCP
HTTP/2	Dạng nhị phân, nhiều luồng trên một kết nối	TCP
HTTP/3	Mất gói ở luồng này ít ảnh hưởng luồng khác	QUIC trên UDP

KHÔNG TRẠNG THÁI
Máy chủ không tự nhớ yêu cầu trước. Cookie giúp duy trì phiên đăng nhập.

HTTPS
HTTP chạy qua TLS, thường dùng cổng 443.

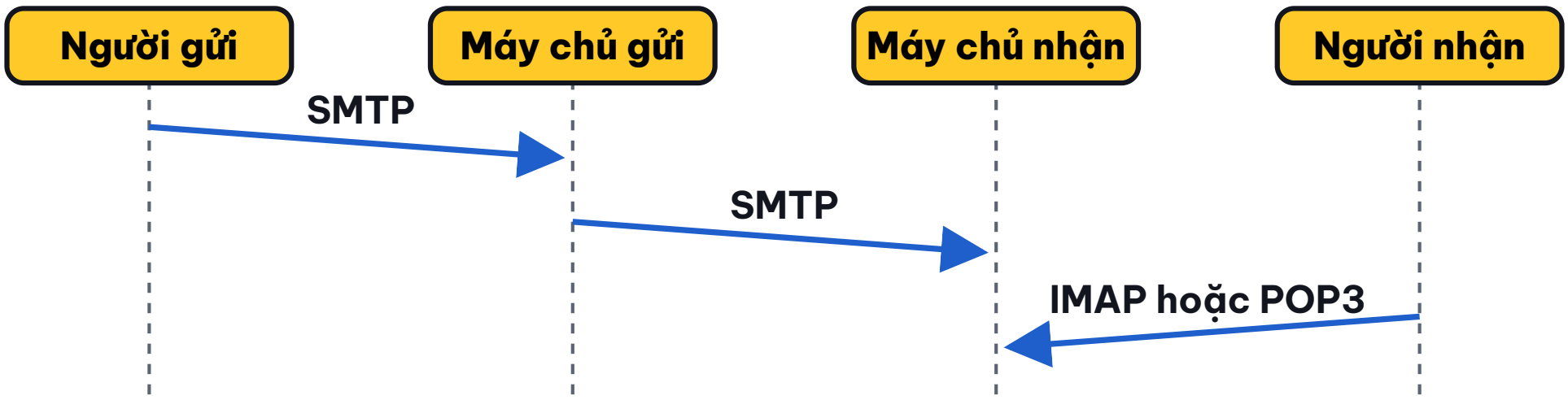
CACHE
Dùng Cache-Control, ETag để giảm tải và tăng tốc.

PROXY VÀ CDN
Đứng giữa để lưu đệm, cân bằng tải, gần người dùng hơn.

Tính toán hay gặp: mỗi đối tượng tốn 2 RTT với HTTP không giữ kết nối, gồm 1 RTT bắt tay và 1 RTT yêu cầu.

70 Email

Gửi thư và đọc thư dùng các giao thức khác nhau.



Giao thức	Cổng	Vai trò
SMTP	25, 587	Gửi và chuyển thư
POP3	110, bảo mật 995	Tải thư về máy
IMAP	143, bảo mật 993	Đồng bộ thư trên máy chủ

CHỐNG GIẢ MẠO
SPF, DKIM, DMARC dùng bản ghi DNS để xác minh người gửi.

KHI GỬI THƯ
Máy chủ gửi tra bản ghi MX của tên miền nhận.

71 Các dịch vụ khác

Giao thức	Cổng	Dùng để
FTP	TCP 20, 21	Truyền file, không mã hóa
SFTP	TCP 22	Truyền file qua SSH
TFTP	UDP 69	Truyền file đơn giản, nạp firmware
SSH	TCP 22	Điều khiển từ xa có mã hóa
Telnet	TCP 23	Điều khiển từ xa, không mã hóa
NTP	UDP 123	Đồng bộ thời gian
SNMP	UDP 161, 162	Giám sát thiết bị
Syslog	UDP 514	Gửi nhật ký
RDP	TCP 3389	Điều khiển máy Windows

FTP dùng hai kết nối: cổng 21 cho lệnh, và một kết nối riêng cho dữ liệu.

72 Ba mục tiêu và các mối đe dọa

An toàn thông tin hướng tới 3 mục tiêu CIA: bí mật, toàn vẹn, sẵn sàng.

Tấn công	Cách làm	Phòng chống
Nghe lén	Bắt gói trên đường truyền	Mã hóa: TLS, VPN
Giả mạo IP, ARP	Đóng giả thiết bị khác	DAI, lọc nguồn
Man-in-the-middle	Chen vào giữa hai bên	Chứng chỉ, TLS
DoS, DDoS	Làm quá tải dịch vụ	Giới hạn tốc độ, lọc lưu lượng
Giả mạo DNS	Trả kết quả sai	DNSSEC, DoH
Lừa đảo	Lừa người dùng nhập thông tin	Đào tạo, xác thực 2 lớp

LƯU Ý PHÁP LÝ

Chỉ thử các kỹ thuật trên máy của bạn hoặc phòng lab được phép. Quét, nghe lén, tấn công mạng của người khác có thể vi phạm pháp luật.

73 Mã hóa và băm

Mã hóa đối xứng nhanh nhưng khó trao khóa. Bất đối xứng giải quyết việc trao khóa nhưng chậm hơn. TLS dùng cả hai.

```
import hashlib, hmac

print(hashlib.sha256(b"mang may tinh").hexdigest()[:16])
print(hashlib.sha256(b"Mang may tinh").hexdigest()[:16])

key = b"bi-mat"
print(hmac.new(key, b"chuyen 100k", "sha256").hexdigest()[:16])

p, g = 23, 5                          # Diffie-Hellman với số nhỏ
a, b = 6, 15                          # hai số bí mật
A, B = pow(g, a, p), pow(g, b, p)
print(A, B, pow(B, a, p), pow(A, b, p))
```

KẾT QUẢ KHI CHẠY

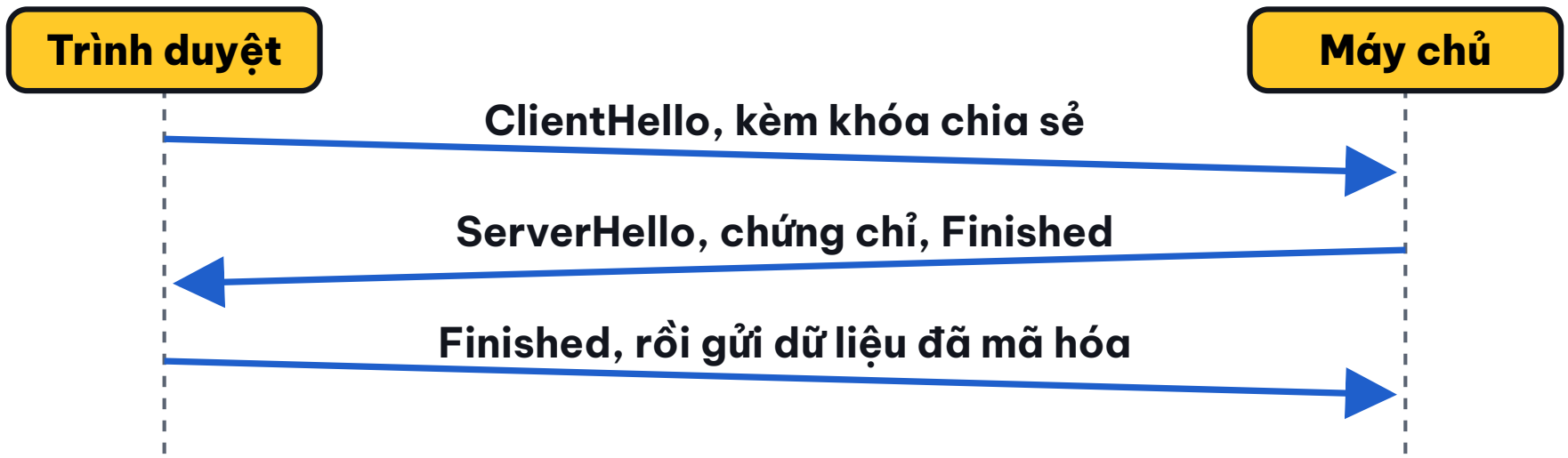
```
4e6bd3a63d0f22ac
a9735285c799c93d
2eb195366d1f0dcc
8 19 2 2
```

Loại	Ví dụ	Mục đích
Đối xứng	AES, ChaCha20	Mã hóa dữ liệu
Bất đối xứng	RSA, ECC	Trao khóa, chữ ký số
Băm	SHA-256	Kiểm tra toàn vẹn
HMAC	HMAC-SHA256	Toàn vẹn và xác thực

Hai số cuối bằng nhau: đó là khóa chung mà không ai gửi trên đường truyền. Số thật phải rất lớn, ví dụ này chỉ để minh họa.

74 TLS và HTTPS

TLS cung cấp mã hóa, toàn vẹn và xác thực máy chủ. TLS 1.3 chỉ cần 1 vòng khứ hồi để bắt tay.



Thành phần	Vai trò
Chứng chỉ	Chứng minh khóa công khai thuộc về tên miền
CA	Tổ chức được tin cậy, ký chứng chỉ
Khóa phiên	Khóa đối xứng dùng cho dữ liệu

- ✓ HTTPS bảo vệ nội dung, nhưng người giữa vẫn có thể thấy địa chỉ IP đích.
- ✓ Có ổ khóa nghĩa là kết nối được mã hóa, không có nghĩa trang web đáng tin.

75 Firewall và ACL

ACL được xét từ trên xuống, gặp dòng khớp đầu tiên thì dừng. Cuối danh sách luôn có một dòng chặn ngầm.

```
rules = [("deny", "192.168.1.50"), ("permit", "192.168.1.")]

def check(src):
    for action, prefix in rules:
        if src.startswith(prefix):
            return action
    return "deny (ngầm định)"

for ip in ("192.168.1.50", "192.168.1.60", "10.0.0.1"):
    print(ip, check(ip))
```

KẾT QUẢ KHI CHẠY

```
192.168.1.50 deny
192.168.1.60 permit
10.0.0.1 deny (ngầm định)
```

```
R1(config)# ip access-list extended WEB-ONLY
R1(config-ext-nacl)# permit tcp 192.168.1.0 0.0.0.255 any eq 443
R1(config-ext-nacl)# deny ip any any
R1(config)# interface gi0/0
R1(config-if)# ip access-group WEB-ONLY in
```

Loại	Lọc theo
ACL chuẩn	Chỉ IP nguồn, nên đặt gần đích
ACL mở rộng	Nguồn, đích, giao thức, cổng, nên đặt gần nguồn
Firewall có trạng thái	Nhớ kết nối, tự cho phép chiều về
IDS, IPS	Phát hiện, và ngăn chặn xâm nhập

76 VPN và IPsec

VPN tạo đường hầm mã hóa qua Internet, để hai mạng hoặc một máy trao đổi dữ liệu như đang ở cùng mạng riêng.



Chế độ tunnel: phần màu vàng được mã hóa cùng phần đuôi ESP.

Thành phần	Ý nghĩa
AH	Xác thực, không mã hóa
ESP	Mã hóa và xác thực
IKE	Thỏa thuận khóa
Tunnel mode	Bọc cả gói IP, dùng giữa hai gateway
Transport mode	Chỉ bảo vệ phần tải

SITE-TO-SITE
Nối hai chi nhánh.

REMOTE ACCESS
Người dùng ở xa vào mạng công ty. WireGuard và OpenVPN là các ví dụ.

77 An toàn tầng 2 và Wi-Fi

Tính năng	Ngăn chặn
Port Security	Thiết bị lạ cắm vào cổng
DHCP Snooping	Máy chủ DHCP giả
Dynamic ARP Inspection	Giả mạo ARP
802.1X	Thiết bị chưa xác thực vào mạng

```
Switch(config)# interface fa0/1
Switch(config-if)# switchport mode access
Switch(config-if)# switchport port-security
Switch(config-if)# switchport port-security maximum 1
Switch(config-if)# switchport port-security violation shutdown
```

Chuẩn Wi-Fi	Đánh giá
WEP	Đã bị phá, không dùng
WPA	Cũ, nên tránh
WPA2	Dùng AES, vẫn rất phổ biến
WPA3	Dùng SAE, chống dò mật khẩu ngoại tuyến tốt hơn

Trong trường học và doanh nghiệp, chế độ Enterprise xác thực từng người dùng qua 802.1X và RADIUS, tốt hơn dùng chung một mật khẩu.

78 Lệnh mạng cần thuộc

Việc cần làm	Windows	Linux
Xem IP, gateway, DNS	`ipconfig /all`	`ip a`, `ip route`
Kiểm tra thông mạng	`ping 8.8.8.8`	`ping 8.8.8.8`
Xem đường đi	`tracert vnnic.vn`	`traceroute vnnic.vn`
Tra DNS	`nslookup vnnic.vn`	`dig vnnic.vn`
Xem bảng ARP	`arp -a`	`ip neigh`
Xem kết nối, cổng	`netstat -ano`	`ss -tulpn`
Xem bảng định tuyến	`route print`	`ip route`
Xóa cache DNS	`ipconfig /flushdns`	Tùy bản phân phối
Thử web, API	`curl -I URL`	`curl -I URL`

Bài tập: chạy lần lượt các lệnh trên máy bạn, ghi lại IP, gateway, DNS, rồi giải thích từng giá trị.

Quy trình tìm lỗi mạng

Đi từ dưới lên theo từng tầng. Mỗi bước loại trừ một nhóm nguyên nhân.

1 **Kết nối vật lý**
Cáp, đèn cổng, Wi-Fi đã kết nối chưa?

2 **Địa chỉ IP**
`ipconfig` : thấy 169.254.x.x là chưa nhận được DHCP.

3 **Ping gateway**
Không được thì lỗi nằm trong mạng nội bộ.

4 **Ping 8.8.8.8**
Không được thì lỗi ở router, hoặc đường ra Internet.

5 **Ping theo tên miền**
IP được mà tên không được thì lỗi DNS.

6 **Thử dịch vụ**
Dùng `curl` , kiểm tra cổng, firewall, proxy.

Sau khi sửa, ghi lại nguyên nhân và cách làm. Đây là thói quen rất giá trị khi đi làm.

79 Wireshark

Công cụ bắt và phân tích gói tin, cách nhanh nhất để nhìn thấy các giao thức đã học.

Bộ lọc hiển thị	Tác dụng
`ip.addr == 192.168.1.10`	Gói liên quan một IP
`tcp.port == 443`	Lưu lượng cổng 443
`dns`	Truy vấn DNS
`arp`, `icmp`, `dhcp`	Theo giao thức
`tcp.flags.syn == 1`	Gói mở kết nối
`tcp.analysis.retransmission`	Gói gửi lại

1. **Thử bắt tay TCP**
   Lọc theo IP, tìm SYN, SYN-ACK, ACK và đối chiếu số seq, ack.

2. **Thử DNS**
   Chạy `nslookup`, xem truy vấn và bản ghi trả về.

3. **Thử ping**
   Xem cặp ICMP Echo Request và Echo Reply.

Chỉ bắt gói trên máy của bạn hoặc mạng bạn được phép.

80 Làm quen Cisco IOS

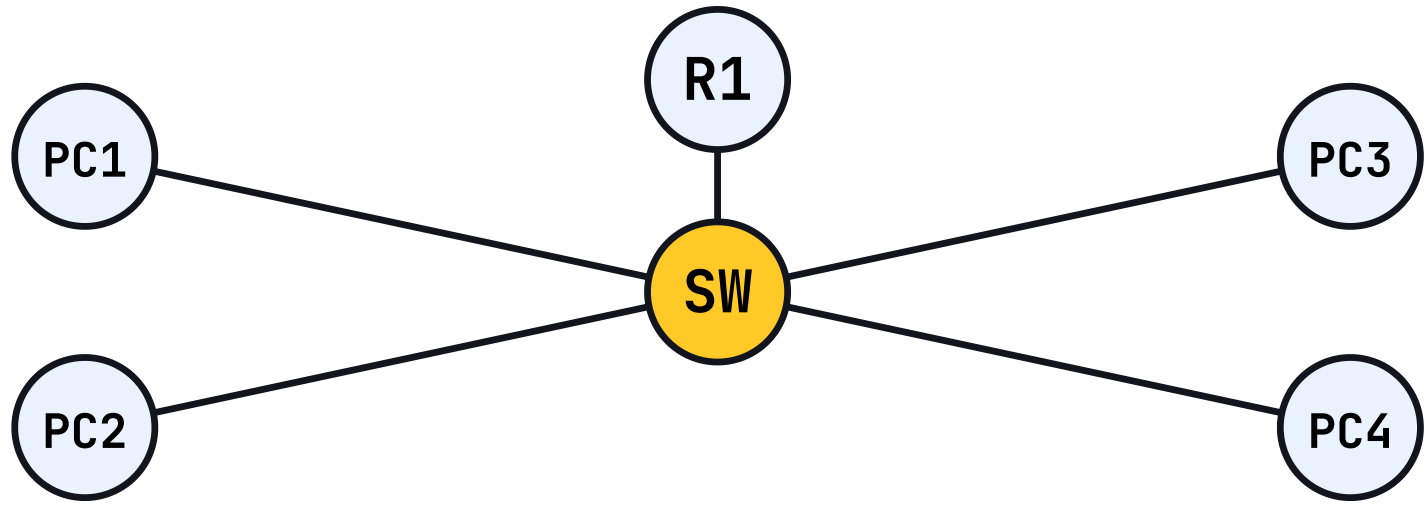
Packet Tracer của Cisco là công cụ mô phỏng phổ biến cho thực hành mạng. Các lệnh dưới đây dùng được cho phần lớn bài lab.

```
Router> enable
Router# configure terminal
Router(config)# hostname R1
R1(config)# enable secret MatKhauManh
R1(config)# interface gi0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# end
R1# show ip interface brief
R1# copy running-config startup-config
```

Dấu nhắc	Chế độ
`Router>`	Người dùng, chỉ xem cơ bản
`Router#`	Đặc quyền, xem cấu hình
`Router(config)#`	Cấu hình toàn cục
`Router(config-if)#`	Cấu hình một cổng

- ✓ Gõ `?` để xem lệnh gợi ý, Tab để gõ nhanh.
- ✓ Quên `no shutdown` là lỗi hay gặp nhất với cổng router.
- ✓ Tên cổng tùy dòng thiết bị: fa0/0, gi0/0, gi0/0/0.
- ✓ Không lưu cấu hình thì khởi động lại sẽ mất.

Lab 1: Mạng LAN và Lab 2: VLAN



LAB 1: LAN CƠ BẢN

Nối PC1, PC2 vào switch. Đặt 192.168.1.10/24 và 192.168.1.11/24. Ping lẫn nhau, rồi xem `show mac address-table`.

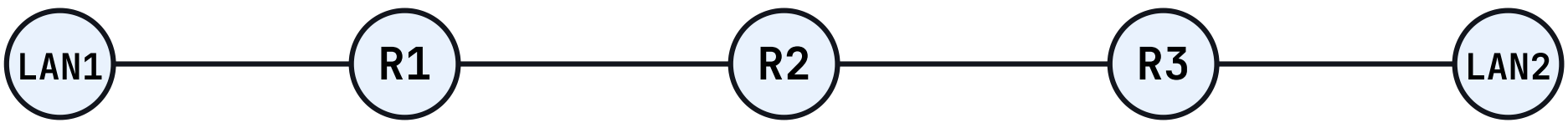
LAB 2: VLAN VÀ ROUTER ON A STICK

Tạo VLAN 10 (PC1, PC3) và VLAN 20 (PC2, PC4). Cổng nối R1 đặt trunk. Tạo subinterface .10, .20 trên R1. Mỗi PC đặt gateway đúng VLAN.

Máy	VLAN	IP	Gateway
PC1	10	192.168.10.11/24	192.168.10.1
PC3	10	192.168.10.12/24	192.168.10.1
PC2	20	192.168.20.11/24	192.168.20.1
PC4	20	192.168.20.12/24	192.168.20.1

- ✓ Kiểm tra: PC1 ping được PC3 mà không qua router.
- ✓ Kiểm tra: PC1 ping được PC2 khi router đã chạy đúng.
- ✓ Thử tắt subinterface .20 và giải thích kết quả.

Lab 3: Định tuyến và Lab 4: DHCP, NAT



Đoạn	Mạng
LAN1	192.168.1.0/24
R1 và R2	10.0.0.0/30
R2 và R3	10.0.0.4/30
LAN2	192.168.2.0/24

LAB 3: ĐỊNH TUYẾN
Lần 1 dùng tuyến tĩnh trên cả 3 router. Lần 2 xóa tuyến tĩnh, chạy OSPF area 0. So sánh `show ip route` ở hai lần.

```
R1(config)# ip nat inside source list 1 interface gi0/1 overload
R1(config)# access-list 1 permit 192.168.1.0 0.0.0.255
R1(config)# interface gi0/0
R1(config-if)# ip nat inside
R1(config)# interface gi0/1
R1(config-if)# ip nat outside
R1# show ip nat translations
```

LAB 4: DHCP VÀ NAT
Cấu hình R1 cấp DHCP cho LAN1 và làm PAT ra một router giả lập ISP. Kiểm tra bảng NAT khi PC ping ra ngoài.

Các lệnh cần chỉnh theo tên cổng trong sơ đồ lab của bạn.

Thiết kế mạng ký túc xá

Đề: khu ký túc xá có 900 sinh viên, 200 cán bộ, 400 khách Wi-Fi, 30 server, 20 thiết bị quản trị. Dùng dải 10.10.0.0/16.

```python
import ipaddress as ip
from math import ceil, log2

needs = [("SinhVien", 10, 900), ("CanBo", 20, 200), ("Khach", 30, 400),
         ("Server", 40, 30), ("QuanTri", 99, 20)]

addr = int(ip.ip_address("10.10.0.0"))
for name, vlan, hosts in sorted(needs, key=lambda x: -x[2]):
    bits = ceil(log2(hosts + 2))
    net = ip.ip_network((addr, 32 - bits))
    print(f"VLAN {vlan:<3} {name:<9} {str(net):<15} GW {net[1]}")
    addr += 2 ** bits
```

KẾT QUẢ KHI CHẠY

```
VLAN 10  SinhVien  10.10.0.0/22    GW 10.10.0.1
VLAN 30  Khach     10.10.4.0/23    GW 10.10.4.1
VLAN 20  CanBo     10.10.6.0/24    GW 10.10.6.1
VLAN 40  Server    10.10.7.0/27    GW 10.10.7.1
VLAN 99  QuanTri   10.10.7.32/27   GW 10.10.7.33
```

Quyết định	Lý do
Tách VLAN theo nhóm	Giảm quảng bá, dễ đặt chính sách
Chặn khách vào VLAN Server	Giảm rủi ro an toàn
Hai đường lên lõi	Một đường hỏng vẫn còn mạng
Chừa dải dự phòng	Dễ mở rộng sau này
Dùng WPA2/3 Enterprise	Xác thực theo từng người

Bài tập: vẽ sơ đồ, thêm VLAN cho camera 150 thiết bị, viết ACL cho khách chỉ ra Internet.

Kiến trúc mạng hiện đại

Kiến trúc	Ý tưởng	Dùng ở
Ba lớp	Lõi, phân phối, truy nhập	Doanh nghiệp, trường học
Spine-leaf	Mọi leaf nối tới mọi spine	Trung tâm dữ liệu
SDN	Tách phần điều khiển khỏi phần chuyển gói	Mạng lớn, tự động hóa
Mạng đám mây	VPC, subnet, bảng tuyến được định nghĩa bằng phần mềm	AWS, Azure, Google Cloud
5G	Băng rộng di động, độ trễ thấp hơn	Điện thoại, IoT

Kỹ sư mạng ngày nay cần thêm tự động hóa: REST API, JSON, Python, Ansible, Terraform.

Bài mẫu: subnet

Đề 1. Tìm địa chỉ mạng, quảng bá, dải máy của 172.16.35.123/20.

Lời giải. /20 nằm ở octet 3, mặt nạ 255.255.240.0, bước nhảy 16. 35 nằm giữa 32 và 48. Mạng 172.16.32.0, quảng bá 172.16.47.255.

Đề 2. Chia 10.0.0.0/8 thành ít nhất 1000 mạng con. Prefix mới là gì, mỗi mạng có bao nhiêu máy?

Lời giải. $2^{10} = 1024$ nên mượn 10 bit, thành /18. Còn 14 bit phần máy, được $2^{14} - 2$ máy.

Đề 3. Hai máy 192.168.5.33/27 và 192.168.5.65/27 có cùng mạng không?

Lời giải. Bước nhảy 32. Một máy thuộc mạng .32, máy kia thuộc mạng .64. Khác mạng.

```python
import ipaddress as ip

n = ip.ip_interface("172.16.35.123/20").network
print(n, n.broadcast_address, n[1], n[-2])

print(2 ** 14 - 2)

a = ip.ip_interface("192.168.5.33/27").network
b = ip.ip_interface("192.168.5.65/27").network
print(a, b, a == b)
```

KẾT QUẢ KHI CHẠY

```
172.16.32.0/20 172.16.47.255 172.16.32.1 172.16.47.254
16382
192.168.5.32/27 192.168.5.64/27 False
```

Bài mẫu: trễ và truyền file

Đề 1. File 10 MB gửi qua đường 20 Mbps, dài 2.000 km, tốc độ tín hiệu 2×10^8 m/s. Tính thời gian tới khi bit cuối cùng tới đích.

Lời giải. Truyền = 80 Mbit / 20 Mbps = 4 giây. Lan truyền = 10 ms. Tổng 4,01 giây.

Đề 2. Gửi 4 gói, mỗi gói 1.500 byte, qua 3 chặng 10 Mbps. Bỏ qua lan truyền và xử lý. Tính thời gian.

Lời giải. Mỗi gói mất 1,2 ms một chặng. Tổng (4 + 3 − 1) × 1,2 = 7,2 ms.

Đề 3. Tính tích băng thông và độ trễ của đường 1 Gbps có độ trễ một chiều 50 ms.

Lời giải. $10^9 \times 0{,}05 = 50$ triệu bit đang nằm trên đường truyền.

```
size = 10e6 * 8
print(round(size / 20e6 + 2000e3 / 2e8, 3), "giây")

t = 1500 * 8 / 10e6
print(round((4 + 3 - 1) * t * 1000, 2), "ms")

print(int(1e9 * 0.05), "bit")
```

KẾT QUẢ KHI CHẠY

```
4.01 giây
7.2 ms
50000000 bit
```

Bài mẫu: TCP

Đề 1. Máy A gửi hai đoạn liên tiếp: seq 90 dài 20 byte, seq 110 dài 40 byte. Đoạn đầu mất, đoạn sau tới nơi. B gửi ACK số mấy?

Lời giải. ACK 90. TCP xác nhận tích lũy, B vẫn đang chờ byte 90.

Đề 2. Với Reno, cwnd đang 24 MSS thì nhận 3 ACK trùng. Tính ssthresh và cwnd mới.

Lời giải. ssthresh = 12, cwnd = 12 theo mô hình rút gọn. Nếu là hết giờ thì cwnd = 1.

Đề 3. Cửa sổ 64 KB, RTT 40 ms. Thông lượng tối đa bao nhiêu?

Lời giải. Khoảng 13,1 Mbps, tính 64 KB là 65.536 byte.

Đề 4. Slow start từ cwnd = 1 MSS, ssthresh = 16. Sau bao nhiêu RTT thì cwnd đạt 16?

Lời giải. 1, 2, 4, 8, 16 nên cần 4 RTT.

```
print(round(65536 * 8 / 0.04 / 1e6, 1), "Mbps")

cwnd, rtt = 1, 0
while cwnd < 16:
    cwnd, rtt = cwnd * 2, rtt + 1
print(rtt, "RTT")
```

KẾT QUẢ KHI CHẠY

```
13.1 Mbps
4 RTT
```

Tự luyện: subnet và IPv6

Tự làm bằng tay trước. Đáp án ở trang sau.

BÀI 1 ĐẾN 10
Tìm địa chỉ mạng, địa chỉ quảng bá và số máy dùng được của mỗi địa chỉ.

Bài	Địa chỉ	Bài	Địa chỉ
1	10.20.30.40/13	6	192.0.2.77/28
2	172.19.200.5/21	7	10.200.5.9/11
3	192.168.7.100/29	8	172.16.1.254/23
4	10.0.255.1/15	9	192.168.100.33/30
5	172.31.0.200/25	10	10.10.10.10/17

BÀI 11: VLSM
Chia 172.16.0.0/24 cho 4 mạng cần 60, 30, 12 và 2 máy.

BÀI 12: CHIA ĐỀU
Chia 10.1.0.0/16 thành ít nhất 6 mạng con bằng nhau. Cho biết prefix mới và mạng con thứ 3.

BÀI 13: TỔNG HỢP TUYẾN
Gộp 192.168.8.0/24 đến 192.168.15.0/24 thành một tuyến.

BÀI 14: IPV6
Rút gọn 2001:0db8:0000:0042:0000:0000:0000:0001.

BÀI 15: SO MẠNG
Máy 10.1.1.200/25 có gửi trực tiếp được cho 10.1.1.100/25 không?

Đáp án tự luyện

Bài 1 đến 10 ghi theo thứ tự: số bài, địa chỉ mạng, địa chỉ quảng bá, số máy. Các dòng sau là bài 11 đến 15. Đáp án do code tính ra.

KẾT QUẢ KHI CHẠY

```
1   10.16.0.0       10.23.255.255     524286
2   172.19.200.0    172.19.207.255    2046
3   192.168.7.96    192.168.7.103     6
4   10.0.0.0        10.1.255.255      131070
5   172.31.0.128    172.31.0.255      126
6   192.0.2.64      192.0.2.79        14
7   10.192.0.0      10.223.255.255    2097150
8   172.16.0.0      172.16.1.255      510
9   192.168.100.32  192.168.100.35    2
10  10.10.0.0       10.10.127.255     32766
60 172.16.0.0/26
30 172.16.0.64/27
12 172.16.0.96/28
2 172.16.0.112/30
10.1.64.0/19
192.168.8.0/21
2001:db8:0:42::1
10.1.1.128/25 10.1.1.0/25
```

Bài 12: cần 3 bit nên prefix mới là /19. Bài 15: hai máy khác mạng nên phải qua router.

Trắc nghiệm (1/2)

1. Thiết bị nào chia miền quảng bá?
A. Hub B. Switch C. Router D. Repeater

2. PDU của TCP gọi là gì?
A. Bit B. Khung C. Gói tin D. Đoạn

3. Mặt nạ của /27 là?
A. 255.255.255.192 B. 255.255.255.224 C. 255.255.255.240 D. 255.255.255.248

4. Mạng /28 có bao nhiêu máy dùng được?
A. 14 B. 16 C. 30 D. 6

5. Địa chỉ nào thuộc dải riêng RFC 1918?
A. 172.32.1.1 B. 11.0.0.1 C. 192.168.50.1 D. 169.254.1.1

6. 192.168.1.77/26 thuộc mạng nào?
A. 192.168.1.0 B. 192.168.1.64 C. 192.168.1.76 D. 192.168.1.128

7. Giao thức nào tìm MAC từ IPv4?
A. DNS B. DHCP C. ARP D. ICMP

8. Thứ tự đúng của DHCP?
A. Discover, Offer, Request, ACK B. Offer, Discover, ACK, Request C. Request, Offer, Discover, ACK D. Discover, Request, Offer, ACK

9. HTTPS thường dùng cổng nào?
A. 80 B. 22 C. 443 D. 53

10. Giao thức nào không cần thiết lập kết nối?
A. TCP B. UDP C. SSH D. HTTP/1.1

Trắc nghiệm (2/2)

11. Trường nào của IPv4 giảm sau mỗi router?
A. Total Length B. TTL C. Protocol D. Identification

12. Gói tới 10.1.1.5, bảng có 4 tuyến. Chọn tuyến nào?
A. 0.0.0.0/0 B. 10.0.0.0/8 C. 10.1.0.0/16 D. 10.1.1.0/24

13. AD mặc định của OSPF trên Cisco?
A. 1 B. 90 C. 110 D. 120

14. Dạng rút gọn đúng của 2001:0db8:0:0:0:0:0:1?
A. 2001:db8::1 B. 2001:db8:1 C. 2001::db8::1 D. 2001:db8:::1

15. Cờ nào dùng để mở kết nối TCP?
A. FIN B. RST C. PSH D. SYN

16. Cơ chế nào bảo vệ bộ đệm bên nhận?
A. Kiểm soát luồng B. Định tuyến C. NAT D. Phân mảnh

17. cwnd = 16 thì nhận 3 ACK trùng. ssthresh mới là?
A. 4 B. 8 C. 16 D. 32

18. Bản ghi DNS nào chỉ máy chủ nhận thư?
A. A B. NS C. MX D. PTR

19. Giao thức nào quản trị từ xa có mã hóa?
A. Telnet B. SSH C. FTP D. TFTP

20. Cuối mỗi ACL Cisco có dòng ngầm định nào?
A. permit any B. deny any C. permit tcp D. Không có gì

Đáp án trắc nghiệm

Câu	Đáp án	Giải thích
1	C	Router không chuyển quảng bá sang mạng khác
2	D	Đoạn (segment)
3	B	3 bit mượn ở octet cuối: 224
4	A	$2^4 - 2 = 14$
5	C	172.32 ngoài dải, 169.254 là link-local
6	B	Bước nhảy 64, 77 nằm từ 64 đến 127
7	C	ARP
8	A	DORA
9	C	443
10	B	UDP không bắt tay
11	B	TTL
12	D	Prefix dài nhất
13	C	110
14	A	:: chỉ dùng một lần
15	D	SYN
16	A	Trường Window
17	B	Một nửa cwnd
18	C	MX
19	B	SSH
20	B	Chặn mọi thứ chưa được cho phép

20 câu vấn đáp (1/2)

1. OSI khác TCP/IP thế nào?
OSI có 7 tầng, là mô hình tham chiếu. TCP/IP đơn giản hơn và là nền của Internet.

2. Switch khác router thế nào?
Switch chuyển khung trong mạng theo MAC. Router chuyển gói giữa các mạng theo IP.

3. MAC khác IP thế nào?
MAC dùng trong một chặng. IP dùng để định danh và định tuyến giữa các mạng.

4. Default gateway là gì?
Router mà máy gửi gói tới khi đích nằm ngoài mạng của mình.

5. Vì sao cần subnet?
Giảm quảng bá, dùng địa chỉ hiệu quả, tách vùng để quản lý và bảo mật.

6. VLAN giúp gì?
Chia miền quảng bá theo logic mà không cần thêm switch vật lý.

7. Vì sao mạng có vòng ở tầng 2 lại nguy hiểm?
Khung không có TTL nên quảng bá chạy mãi, gây bão quảng bá.

8. NAT giải quyết vấn đề gì?
Cho nhiều máy dùng địa chỉ riêng đi ra Internet qua ít địa chỉ công cộng.

9. Vì sao cần IPv6?
IPv4 cạn địa chỉ. IPv6 có không gian rất lớn và đơn giản hóa một số xử lý.

10. Ping dùng giao thức gì?
ICMP Echo Request và Echo Reply.

20 câu vấn đáp (2/2)

11. RIP khác OSPF thế nào?
RIP dùng số hop, vector khoảng cách. OSPF dùng cost, trạng thái liên kết, hội tụ nhanh hơn.

12. Vì sao Internet dùng BGP?
Các AS cần trao đổi tuyến theo chính sách riêng, ở quy mô toàn cầu.

13. TCP đảm bảo tin cậy bằng cách nào?
Số thứ tự, ACK, gửi lại, checksum, kiểm soát luồng và tắc nghẽn.

14. Kiểm soát luồng khác kiểm soát tắc nghẽn thế nào?
Một bên bảo vệ bên nhận, một bên bảo vệ mạng.

15. Vì sao DNS hay dùng UDP?
Truy vấn ngắn, cần nhanh, không tốn bắt tay. Phản hồi lớn có thể chuyển sang TCP.

16. HTTP khác HTTPS thế nào?
HTTPS là HTTP chạy qua TLS, có mã hóa và xác thực máy chủ.

17. Mã hóa đối xứng khác bất đối xứng thế nào?
Đối xứng dùng một khóa và nhanh. Bất đối xứng dùng cặp khóa, chậm hơn.

18. Firewall có trạng thái là gì?
Firewall nhớ kết nối đã mở, nên tự cho phép gói phản hồi hợp lệ.

19. Máy nhận 169.254.x.x thì sao?
Nó không nhận được DHCP và tự gán địa chỉ link-local.

20. Mở một trang web thì những gì xảy ra?
DNS, ARP, TCP, TLS, HTTP, định tuyến, phản hồi và hiển thị.

Lỗi hay mất điểm

Lỗi	Cách đúng
Nhầm bit và byte	Đổi cùng đơn vị trước khi tính
Nhầm truyền, lan truyền	L / R khác d / s
Quên trừ 2 địa chỉ	Số máy = 2^h - 2
VLSM cấp mạng nhỏ trước	Cấp mạng lớn trước
Offset phân mảnh tính theo byte	Chia cho 8
ACK là byte vừa nhận	ACK là byte tiếp theo cần
Dùng dB trong Shannon	Đổi dB sang tỉ số
Chọn tuyến theo metric trước	Prefix dài nhất trước
Dùng hai dấu :: trong IPv6	Chỉ được dùng một lần

Tra nhanh: cổng và giao thức

Cổng	Dịch vụ
20, 21	FTP
22	SSH, SFTP
23	Telnet
25, 587	SMTP
53	DNS
67, 68	DHCP
69	TFTP
80	HTTP
110	POP3
123	NTP

Cổng	Dịch vụ
143	IMAP
161, 162	SNMP
179	BGP
443	HTTPS
514	Syslog
520	RIP
993	IMAPS
995	POP3S
3306	MySQL
3389	RDP

Số Protocol trong IPv4	Giao thức
1	ICMP
6	TCP
17	UDP
50	ESP
89	OSPF

Tra nhanh: chuẩn và RFC

RFC là tài liệu kỹ thuật gốc của Internet, do IETF công bố. Khi sách và RFC mâu thuẫn, hãy đọc RFC mới nhất.

Giao thức	Chuẩn	Giao thức	Chuẩn
IPv4	RFC 791	**DNS**	RFC 1034, 1035
IPv6	RFC 8200	**DHCP**	RFC 2131
ICMP	RFC 792	**NAT**	RFC 3022
ARP	RFC 826	**Dải riêng**	RFC 1918
TCP	RFC 9293	**OSPFv2**	RFC 2328
UDP	RFC 768	**BGP-4**	RFC 4271
RTO của TCP	RFC 6298	**HTTP**	RFC 9110
QUIC	RFC 9000	**HTTP/3**	RFC 9114
TLS 1.3	RFC 8446	**Ethernet**	IEEE 802.3
Wi-Fi	IEEE 802.11	**VLAN**	IEEE 802.1Q

Tra cứu RFC tại rfc-editor.org. Một số RFC đã có bản cập nhật hoặc thay thế.

Kế hoạch 15 tuần

Tuần	Nội dung	Làm
1	Tổng quan, trễ, OSI	Bài tập tính trễ
2	Tầng vật lý, cáp, thiết bị	Bấm cáp, nhận diện thiết bị
3	Phát hiện lỗi, truy nhập kênh	Tính CRC, Hamming
4	Ethernet, ARP, switch	Lab 1, Wireshark ARP
5	VLAN, STP, Wi-Fi	Lab 2
6	IPv4, subnet	Giải 30 bài subnet
7	VLSM, DHCP, NAT	Lab 4
8	IPv6, ôn giữa kỳ	Trắc nghiệm phần đầu
9	Định tuyến tĩnh, Dijkstra	Lab 3 lần 1
10	RIP, OSPF, BGP	Lab 3 lần 2
11	UDP, TCP, bắt tay	Wireshark TCP
12	Tắc nghẽn, socket	Viết chat bằng socket
13	DNS, HTTP, email	Wireshark DNS, HTTP
14	An toàn mạng	ACL, port security
15	Ôn tập	Bài mẫu, trắc nghiệm, vấn đáp

Trình tự bám theo cẩm nang. Nếu trường bạn học từ tầng ứng dụng xuống, hãy đổi thứ tự cho khớp.

Đi tiếp lên CCNA

CCNA là chứng chỉ mạng căn bản phổ biến của Cisco. Bảng dưới theo khung kiến thức v1.1.

Mảng kiến thức	Tỉ trọng	Xem ở
Nền tảng mạng	20%	Phần 1 đến 4
Truy nhập mạng	20%	Phần 3
Kết nối IP	25%	Phần 5
Dịch vụ IP	10%	Phần 4, 7
An toàn cơ bản	15%	Phần 8
Tự động hóa	10%	Tự học thêm

Cẩm nang này đủ làm nền, nhưng chưa thay thế tài liệu luyện CCNA. Hãy xem khung kiến thức mới nhất trên trang Cisco trước khi ôn.

Tài liệu nên đọc

Nguồn	Vì sao nên đọc
Kurose, Ross: Computer Networking	Giáo trình phổ biến, học từ trên xuống
Tanenbaum: Computer Networks	Trình bày rộng và sâu
Stallings: Data and Computer Communications	Mạnh về truyền dữ liệu, tầng dưới
RFC Editor	Tài liệu gốc của giao thức Internet
Cisco Networking Academy	Packet Tracer và khóa thực hành
Wireshark User's Guide	Hướng dẫn phân tích gói tin
VNNIC	Thông tin Internet Việt Nam, tên miền, IPv6

1 **Bắt đầu nhỏ**
Tự dựng mạng trong Packet Tracer, không chỉ đọc lý thuyết.

2 **Nhìn gói tin thật**
Mỗi giao thức học xong, hãy bắt thử bằng Wireshark.

Thấy chỗ nào chưa đúng? Anh em comment giúp để Code4Life sửa ở bản sau nhé.

Biên soạn: Tào Việt Đức, Code4Life Technology, với sự hỗ trợ của AI. Code Python đã chạy thử trên Python 3.14. Các lệnh Cisco cần thử lại trên thiết bị hoặc Packet Tracer của bạn.